

Quantization-Aware Design Space Exploration of Fixed Point Reciprocal Approximation Architectures on Artix-7 FPGAs

NAVANEETH KUTTY*, ANISH MURALI*, and PRIYAN M S K*, Department of Electronics & Communication Engineering, SSN College of Engineering, India

Fixed-point reciprocal computation is an important operation in FPGA-based arithmetic pipelines, where division is often replaced by dedicated approximation methods to reduce computational cost. This work presents a comparative evaluation of five reciprocal approximation architectures - Direct Lookup Table (LUT), LUT with Linear Interpolation, Newton - Raphson iteration, Goldschmidt iteration, and a hybrid LUT-Interpolation/Newton - Raphson design combining linear interpolation with a single Newton-Raphson refinement iteration - implemented in Q16.16 and Q8.24 fixed-point formats for a Xilinx Artix-7 FPGA (Nexys A7-100T) over the input range $[0.5, 2.5]$ using one million verification vectors. A Pareto analysis based on maximum relative error, latency, and energy per operation is used to characterize the trade-offs among the evaluated architectures. In Q8.24, the Direct LUT provides the lowest latency (12.9 ns) and energy per operation (0.346 nJ/op), with a maximum relative error of 0.098%, while Newton-Raphson provides the lowest error ($1.5 \times 10^{-5}\%$) at a latency of 178.0 ns and an energy consumption of 1.190 nJ/op. Goldschmidt achieves comparable numerical accuracy with a 22% reduction in latency relative to Newton-Raphson at a marginally higher energy consumption, while the hybrid architecture provides an intermediate trade-off between numerical accuracy, latency, and energy consumption. The resulting hardware measurements provide a unified reference for evaluating fixed-point reciprocal architectures under different precision, latency, and energy constraints, with relevance to FPGA-based signal processing, normalization, coordinate transformation, and other division-free arithmetic pipelines.

CCS Concepts: • **Hardware** → *Hardware accelerators*; **Arithmetic and datapath circuits**; **Reconfigurable logic applications**.

Additional Key Words and Phrases: Reciprocal approximation, FPGA, Fixed-point arithmetic, Quantization, Newton-Raphson method, Goldschmidt algorithm, Interpolation, Pareto Analysis

ACM Reference Format:

Navaneeth Kutty, Anish Murali, and Priyan M S K. 2018. Quantization-Aware Design Space Exploration of Fixed Point Reciprocal Approximation Architectures on Artix-7 FPGAs. 1, 1 (September 2018), 35 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Division and reciprocal computation are comparatively expensive arithmetic operations to implement efficiently on field-programmable gate arrays (FPGAs). Unlike addition and multiplication, which map efficiently onto dedicated FPGA arithmetic resources, division typically has no equivalent dedicated hardware unit for efficient acceleration. Consequently, hardware architectures often avoid direct division or replace it with reciprocal approximation followed

* All authors contributed equally to this research.

Authors' Contact Information: Navaneeth Kutty, navaneeth2310165@ssn.edu.in; Anish Murali, anish2310158@ssn.edu.in; Priyan M S K, priyan2310182@ssn.edu.in, Department of Electronics & Communication Engineering, SSN College of Engineering, Chennai, Tamil Nadu, India.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

by multiplication [1]. Efficient reciprocal computation is therefore important in applications where division occurs frequently or lies within an iterative processing path.

Beyond direct division, digit-recurrence, table-lookup, and functional-iteration methods together form the standard toolkit for hardware division and reciprocal computation [15]. Three broad families of reciprocal approximation are commonly considered in hardware implementations. The first relies on direct table lookup, in which the reciprocal of a quantized input is pre-computed offline and stored in on-chip memory, typically block RAM. The second improves upon direct lookup by combining a coarser table with a linear or higher-order interpolation stage, trading additional arithmetic logic for reduced table size and approximation error. The third uses iterative multiplicative refinement, most commonly Newton-Raphson or Goldschmidt iteration, seeded from a coarse initial estimate and refined over one or more pipeline stages. Lookup-based and iterative approaches, as well as combinations of the two, have been widely explored for FPGA-based reciprocal and division computation [1]. Each family occupies a different point in the design space spanned by numerical accuracy, latency, hardware resource cost, and energy consumption, and no single architecture necessarily dominates across all of these dimensions.

A recurring difficulty in comparing these architectures, both in published studies and in practical implementations, is that results are often obtained under different evaluation conditions. Differences in fixed-point format, input range, verification methodology, target device, and reported metrics can make direct architecture-to-architecture comparison difficult. A design that exhibits favorable accuracy or resource utilization under one precision or input distribution may exhibit different behavior under another. Furthermore, synthesis-only estimates may not accurately reflect the timing and resource characteristics obtained after place-and-route.

This work addresses the difficulty of making such comparisons by evaluating five reciprocal approximation architectures—Direct LUT, LUT with Linear Interpolation, Newton-Raphson iteration, Goldschmidt iteration, and a hybrid LUT-Interpolation/Newton-Raphson design—under an identical verification methodology, input range, and target platform. All five architectures are implemented in both Q8.24 and Q16.16 fixed-point formats, synthesized and implemented for a Xilinx Artix-7 FPGA (Nexys A7-100T), and validated against one million test vectors uniformly swept over the input domain [0.5, 2.5]. Architecture-specific clock targets are imposed during implementation, with a 2.58-ns target period for the Direct LUT, LUT with Linear Interpolation, Newton-Raphson, and Goldschmidt architectures, and a 3.33-ns target period for the hybrid architecture. Timing closure is treated as a primary design objective. Resource, timing, and power results are derived from post-implementation analysis, while numerical accuracy is evaluated through bit-accurate RTL simulation against a double-precision reference model. The hardware implementations are optimized with timing closure as a primary design objective, with each datapath pipelined and restructured according to its timing-critical paths. Although latency is also evaluated, greater emphasis is placed on achievable clock frequency and timing closure to provide a consistent basis for comparing the architectures.

The remainder of this paper is organized as follows. Section 2 reviews the mathematical background of fixed-point reciprocal approximation and the table-based and iterative approaches that motivate the architectures evaluated in this work. Section 3 describes the design of each of the five evaluated architectures in detail, including the hybrid design’s combination of interpolation and iterative refinement. Section 4 describes the experimental methodology, covering functional verification, timing closure, and power and energy measurement. Section 5 presents the experimental results across resource utilization, timing, numerical accuracy, and power and energy, along with a cross-format comparison between Q8.24 and Q16.16. Section 6 presents a three-objective Pareto trade-off analysis over numerical error, latency, and energy per operation. Section 7 discusses the practical implications, applicability, and limitations of the results, and Section 8 concludes the paper.

1.1 Previous Work and Contributions

Reciprocal computation is a fundamental operation in hardware division and numerical processing. Established hardware approaches include digit-recurrence, table-lookup, functional-iteration, and hybrid architectures, as surveyed in the standard computer-arithmetic textbooks of Ercegovac and Lang [15, 17], Parhami [18], and Muller [19], and in Soderquist and Leaser’s area/performance survey of divide and square-root implementations [16]. Oberman and Flynn provide a widely used classification of these approaches and show that practical implementations often combine multiple algorithmic classes to balance latency, cycle time, and area [1]. Convergence (multiplicative-refinement) division traces to Anderson *et al.*’s IBM System/360 Model 91 floating-point unit [20] and to Flynn’s general treatment of division by functional iteration [21], both of which predate and motivate the Newton-Raphson and Goldschmidt formulations evaluated in this work.

Goldschmidt’s original division-by-convergence method [10] and the classical bipartite ROM reciprocal-table construction of Das Sarma and Matula [12] established, respectively, the iterative-refinement and table-compression foundations on which much of the subsequent hardware reciprocal literature builds, including the LUT-based and Goldschmidt-family architectures evaluated in this work. Subsequent table-lookup-and-addition methods generalized the bipartite idea: Schulte and Stine’s symmetric bipartite tables [23] and de Dinechin and Tisserand’s multipartite table framework [24] unify and extend bipartite table construction to a wider range of elementary functions, while Ercegovac *et al.* [25] and Beuchat and Tisserand [26] propose small-multiplier-based reciprocal, square-root, and division architectures that trade a modest increase in multiplier count for substantially smaller seed tables than a pure LUT approach. Ito *et al.* address the complementary problem of generating a more accurate initial seed for multiplicative refinement through operand modification, reducing the number of refinement iterations required for a given target precision [22]. Piñeiro and Bruguera [27] and Piñeiro *et al.* [28] further develop minimax quadratic table-based interpolation for double-precision reciprocal, square-root, and related elementary-function evaluation, representing a higher-order generalization of the linear interpolation stage used by the LUT-with-interpolation architecture evaluated in this work. Table-based reciprocal computation reduces arithmetic complexity by storing precomputed approximations, but memory requirements increase with precision. Chen *et al.* addressed this trade-off through a compact Taylor-based lookup structure followed by two Newton-Raphson iterations for double-precision reciprocal computation [2]. Their design uses a $2^7 \times 16$ -bit ROM and requires ten clock cycles; an FPGA implementation on a Xilinx Virtex XCV1000E was also reported. Kucukkabak and Akkas similarly combined a lookup-table initial approximation with two Newton-Raphson iterations, demonstrating the established use of table-based seeding to reduce iterative computation [3].

Fixed-point implementations have subsequently explored more hardware-efficient forms of approximation and iteration. Rodríguez-García *et al.* proposed a fixed-point divider based on reciprocal computation using a six-segment second-order polynomial approximation for the initial seed and one Newton-Raphson iteration. Scaling and de-scaling allow the reciprocal calculation to operate over a normalized interval while supporting different fixed-point formats [4]. Libessart *et al.* investigated a pipelined fixed-point reciprocal operator based on Newton-Raphson, further demonstrating the importance of fixed-point representation and hardware pipeline organization [5]. Lunglmayr proposed an FPGA-oriented non-sequential reciprocal approximation based on piecewise linearization with power-of-two node points and a polynomial correction stage, targeting scaling-free, LUT-free implementation for large data-processing pipelines rather than the BRAM-seeded architectures evaluated here [35].

FPGA-oriented comparative work is also available. Kumar, Saha, and Dandapat implemented fixed-point reciprocal and division using both digit-recurrence and multiplicative methods, including Newton-Raphson and Goldschmidt

algorithms. Their designs were implemented in Verilog and synthesized on a Xilinx Virtex-4 FPGA, with frequency, slice utilization, latency, and power evaluated for several operand lengths [6]. Kasim *et al.* investigated another memory-oriented approach in which a precomputed reciprocal value is selected using a reduced number of denominator bits. Their results explicitly demonstrate the exponential trade-off between lookup-table size and approximation error, and the work compares the resulting divider with Goldschmidt and non-restoring implementations [7]. In the floating-point domain, Wang and Nelson compared digit-recurrence and multiplicative division and square-root architectures on Virtex FPGAs [32]. Sorokin evaluated high-speed fixed-point dividers against a vendor IP core baseline [33], and Muñoz *et al.* reported an analogous area/latency/power tradeoff study of Goldschmidt- and Newton-Raphson-based floating-point division and square-root cores [34]; the present work follows a similar comparative methodology but holds the fixed-point format, input domain, and target device fixed across five reciprocal-only architectures rather than varying operator type. Nannarelli and Lang additionally show that power and delay trade off differently across divider radices [31], reinforcing the need for the explicit power/energy characterization performed in Section 5.4 of this work.

Goldschmidt-based fixed-point designs have also been studied specifically for speed and energy efficiency. Kong and Swartzlander proposed a modified Goldschmidt update that achieves faster-than-quadratic (near-cubic) convergence by reusing an intermediate squaring result, reducing the number of refinement iterations needed for a given target precision relative to the standard quadratically convergent formulation used in this work [11]. Ercegovac *et al.* similarly improve Goldschmidt division, square-root, and square-root-reciprocal computation by replacing the final iteration with a looked-up correcting term [29], while Gallagher and Swartzlander address fault tolerance in time-shared triple-modular-redundant Newton-Raphson and Goldschmidt dividers [30]. Paim *et al.* proposed a modified Goldschmidt divider using a suitable denominator approximation and evaluated its convergence and energy characteristics for Q7.8 arithmetic [8]. More recently, Gutiérrez-Ramos *et al.* presented a multifunctional fixed-point architecture combining piecewise-polynomial seed generation with Newton-Raphson iterations for reciprocal, division, square-root, and reciprocal-square-root operations [9]. Their verified hardware configuration uses unsigned Q(16,5,u) and was synthesized on an Intel Stratix-V FPGA.

These studies establish the individual techniques considered in this work, but their implementation conditions vary substantially. Differences in numerical format, precision, input normalization, FPGA family, synthesis tools, architecture, and reported metrics make direct cross-study comparison difficult. Existing comparative work is valuable but generally evaluates a narrower set of methods or focuses on a particular implementation objective rather than examining the combined effects of reciprocal algorithm and fixed-point precision within one common FPGA framework.

1.2 Contributions of This Work

This work presents a systematic post-implementation hardware evaluation of five established reciprocal architectures: **Direct LUT, LUT-based linear interpolation, Newton-Raphson, Goldschmidt, and a hybrid LUT-interpolation/Newton-Raphson approach**. Each architecture is implemented in both **Q8.24 and Q16.16** fixed-point formats, resulting in ten architecture/format configurations evaluated over a common reciprocal input domain.

All implementations are evaluated on the **Xilinx Artix-7 XC7A100T** using architecture-specific clock targets selected according to the timing requirements of each datapath. The Direct LUT, LUT-interpolation, Newton-Raphson, and Goldschmidt architectures are targeted at **2.58 ns (387.6 MHz)**, while the hybrid architecture is targeted at **3.33 ns (300.3 MHz)**. The evaluation considers **numerical accuracy, FPGA resource utilization, timing closure, latency, throughput, post-implementation power, and energy per operation**. Timing closure is assessed independently for each architecture/format combination using post-place-and-route timing analysis. All ten configurations achieve

positive WNS at their respective target periods, providing a consistent basis for comparing the implementation effects of reciprocal architecture and fixed-point precision.

The hardware implementations are pipelined according to their respective datapath structures, with **FPGA BRAM, LUT/FF, and DSP resources** utilized according to the requirements of each architecture. The resulting pipeline depths and achieved clock frequencies are used to characterize the latency and throughput of each configuration. Particular emphasis is placed on the interaction between **arithmetic precision, pipeline structure, resource utilization, and achievable timing**, allowing the comparison to distinguish the effects of fixed-point format from those of the underlying reciprocal architecture.

The contribution is therefore **not the introduction of a new reciprocal algorithm**, but a controlled comparative study of established approaches under a common FPGA implementation environment and two fixed-point precisions. By evaluating **accuracy, resource utilization, timing, latency, power, and energy** under consistent experimental conditions, the study establishes quantitative design points spanning **memory-based approximation, interpolation-based approximation, iterative reciprocal computation, and hybrid architectures**. The resulting comparison provides a common hardware-level basis for examining the trade-offs among these approaches and for understanding how fixed-point precision influences their implementation characteristics.

2 Background

2.1 Fixed-Point Reciprocal Computation

A signed fixed-point number in $QI.F$ format represents a real value v using I integer bits and F fractional bits. Given a stored two's-complement integer representation r , the represented value is

$$v = r \times 2^{-F} \quad (1)$$

where the exponent $-F$ corresponds to the implicit binary point positioned F bits from the least-significant bit. This work evaluates two 32-bit fixed-point configurations: Q8.24, with 8 integer bits and 24 fractional bits, and Q16.16, with 16 integer bits and 16 fractional bits. Q8.24 provides greater fractional precision at the expense of dynamic range, whereas Q16.16 provides a larger integer range with lower fractional precision.

The reciprocal computation considered in this work takes a fixed-point input $x \in [0.5, 2.5]$ and produces a fixed-point output y approximating $1/x$. The bounded input range provides a common operating domain for all evaluated architectures. In a complete reciprocal datapath, arbitrary positive inputs can be range-reduced using leading-bit or exponent normalization before reciprocal evaluation, followed by appropriate re-scaling. This work focuses on the reciprocal evaluation stage and does not include the preceding normalization or subsequent re-scaling logic.

2.2 LUT-Based Approximation

The most direct approach to reciprocal approximation is to precompute $1/x$ for a quantized set of input values and store the results in a lookup table, typically implemented using FPGA block RAM (BRAM). For an N -bit address, a direct LUT requires 2^N entries, with each entry having sufficient width to represent the reciprocal at the desired output precision. The main advantage of this approach is its simple datapath and low latency after pipeline fill. However, increasing the address width causes the memory requirement to grow exponentially, while reducing the address width increases the input quantization error.

LUT-based accuracy can be improved without a proportional increase in table size by adding a linear interpolation stage. Instead of storing the reciprocal at every quantized input value, the table stores reciprocal values at a smaller number of breakpoints, and the value between adjacent breakpoints is estimated using the local slope:

$$y(x) = L[i] + (x - x_i) \times S[i] \quad (2)$$

where x_i is the breakpoint immediately below x , $L[i]$ is the stored reciprocal value at that breakpoint, and $S[i]$ is the slope between adjacent breakpoints. The interpolation stage therefore reduces the required table size at the cost of additional arithmetic logic.

2.3 Iterative Reciprocal Approximation

Iterative methods use a coarse initial estimate followed by one or more refinement stages to improve the reciprocal estimate. Two iterative approaches are considered in this work: Newton-Raphson and Goldschmidt iteration.

Newton-Raphson iteration for the reciprocal function can be derived by finding the root of $f(y) = 1/y - x$. Using the standard Newton update, $y_{n+1} = y_n - f(y_n)/f'(y_n)$, with $f'(y) = -1/y^2$ gives

$$y_{n+1} = y_n \times (2 - x \times y_n) \quad (3)$$

The method converges quadratically when the initial estimate y_0 is sufficiently close to $1/x$. In this work, y_0 is generated using an 8-bit seed LUT and followed by two Newton-Raphson refinement iterations. Each refinement iteration contains two multiplication operations and one subtraction. Thus, the complete Newton-Raphson datapath performs four multiplications and two subtractions per reciprocal estimate.

Goldschmidt iteration uses a related multiplicative refinement process. Given an initial estimate $y_0 \approx 1/x$, the initial values are

$$N_0 = y_0, \quad D_0 = x \times y_0 \quad (4)$$

The refinement factor and subsequent updates are

$$F_i = 2 - D_i, \quad N_{i+1} = N_i \times F_i, \quad D_{i+1} = D_i \times F_i \quad (5)$$

As the iterations proceed, D_i approaches unity and N_i approaches $1/x$. The two multiplication paths can be evaluated in parallel within an iteration, providing greater hardware parallelism than the sequential multiplication structure of Newton-Raphson. This can reduce the critical path at the cost of additional multiplier resources. In this work, the Goldschmidt datapath is evaluated using two refinement iterations, with the N_2 value used as the final reciprocal estimate.

2.4 Theoretical Error Analysis

The measured accuracy differences among the evaluated architectures can be related to the approximation and convergence properties of their underlying methods.

For a Direct LUT with an N -bit input address over the domain $[a, b] = [0.5, 2.5]$, the nominal input quantization interval is

$$\Delta x = \frac{b - a}{2^N}. \quad (6)$$

Since the reciprocal function $f(x) = 1/x$ has its largest magnitude derivative at the lower boundary, $|f'(x)| = 1/x^2$, $|f'(0.5)| = 4$, a first-order estimate of the maximum quantization-induced reciprocal error is

$$e_{\max, \text{LUT}} \approx |f'(0.5)| \Delta x = 4\Delta x. \quad (7)$$

For the 12-bit Direct LUT evaluated in this work, this gives $e_{\max, \text{LUT}} \approx 4 \left(\frac{2}{2^{12}} \right) = 2^{-9} \approx 1.953 \times 10^{-3}$. This estimate is close to the measured maximum absolute errors of 1.950×10^{-3} for Q8.24 and 1.892×10^{-3} for Q16.16 reported in Table 5. The agreement indicates that input-address quantization is the dominant error mechanism for the Direct LUT over the evaluated domain, although the first-order model does not capture the complete effects of address mapping and output quantization.

For the LUT with linear interpolation, the ideal interpolation error is substantially smaller than the direct LUT quantization error. Consequently, the measured interpolation error is primarily influenced by finite-precision representation of the stored interpolation coefficients and intermediate arithmetic rather than by the ideal interpolation remainder.

For the iterative architectures, convergence can be characterized directly using the relative deviation $\epsilon_n = 1 - x \times y_n$. Substituting the Newton-Raphson update from Eq. (3) gives $x \times y_{n+1} = 2(x \times y_n) - (x \times y_n)^2$ and therefore

$$\epsilon_{n+1} = \epsilon_n^2. \quad (8)$$

This is an exact algebraic identity: each refinement iteration squares the current relative deviation. Consequently, the number of correct bits approximately doubles with each iteration while the error remains above the fixed-point quantization floor.

An equivalent error relation applies to Goldschmidt iteration. With $\epsilon_i = 1 - D_i$, the update in Eq. (5) produces the same error-squaring behavior, while the invariant relationship between N_i and D_i preserves the reciprocal estimate. Thus, both iterative methods exhibit quadratic convergence, although their hardware organizations differ.

Equation (8) also explains the accuracy improvement obtained by the hybrid architecture. Its interpolated seed provides a smaller initial deviation ϵ_0 than the coarse LUT seed used by the standalone iterative architectures. The subsequent Newton-Raphson refinement then squares this smaller error, allowing the hybrid architecture to achieve high numerical accuracy with a single refinement stage.

3 Reciprocal Architecture Design

This section describes the hardware design of each of the five reciprocal approximation architectures evaluated in this work. All five architectures share a common input format, output format, and pipelined register discipline (Section 3.6), differing only in their internal datapath. Design parameters for all five architectures, in both evaluated fixed-point formats, are summarized in Table 1.

3.1 Direct LUT

The Direct LUT architecture quantizes the input x to the seed address width (12 bits, per Table 1) and uses the quantized address to directly index a single block-RAM-resident lookup table storing precomputed reciprocal values at full output precision. The datapath consists of an input register, an address-formation stage that extracts the top address-width bits of the normalized input, the BRAM read itself, and an output register. No arithmetic (multiply or add) logic is required in the data path beyond address formation, which is reflected in Table 3's near-zero LUT utilization for this architecture. The trade-off, visible throughout Section 5, is that a 12-bit address table is the coarsest of the five architectures and

Table 1. Design Parameters by Architecture and Fixed-Point Format

Architecture	Format	Int/Frac Bits	Clock Target (ns)	Seed LUT (bits)
Direct LUT	Q8.24	8/24	2.58	12
Direct LUT	Q16.16	16/16	2.58	12
LUT + Interpolation	Q8.24	8/24	2.58	11
LUT + Interpolation	Q16.16	16/16	2.58	11
Newton-Raphson	Q8.24	8/24	2.58	8
Newton-Raphson	Q16.16	16/16	2.58	8
Goldschmidt	Q8.24	8/24	2.58	8
Goldschmidt	Q16.16	16/16	2.58	8
Hybrid	Q8.24	8/24	3.33	9
Hybrid	Q16.16	16/16	3.33	9

consequently exhibits the largest numerical error, concentrated near the lower end of the input domain where $1/x$ changes most rapidly per unit of x .

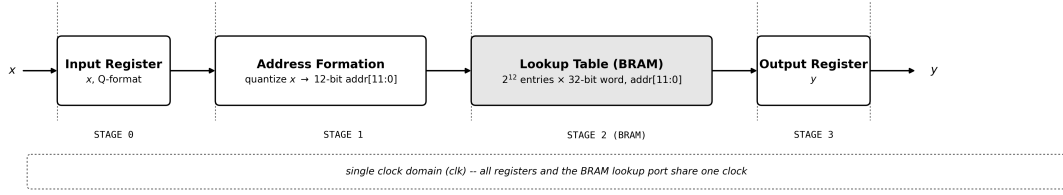


Fig. 1. Direct LUT reciprocal architecture.

3.2 LUT With Linear Interpolation

The interpolation architecture reduces the seed table to 11 address bits and adds a linear interpolation stage per Eq. (2). In addition to the coarser reciprocal-value table, a second table (or a second field of the same table row) stores the local slope $S[i]$ between the current breakpoint and the next. The datapath computes the fractional offset $(x - x_i)$ from the low-order bits of the input not consumed by the address, multiplies this offset by the stored slope, and adds the product to the base table value $L[i]$. This requires one multiplier and one adder beyond the Direct LUT datapath, which is reflected in the non-zero DSP and substantially higher LUT/FF utilization reported in Table 3, but yields more than an order-of-magnitude improvement in maximum relative error relative to the Direct LUT architecture at only one address bit less of table width.

3.3 Newton-Raphson

The Newton-Raphson architecture computes an 8-bit seed estimate y_0 from a coarse BRAM lookup table and applies two Newton-Raphson refinement iterations using Eq. (3). The first iteration computes $y_1 = y_0(2 - xy_0)$, while the second

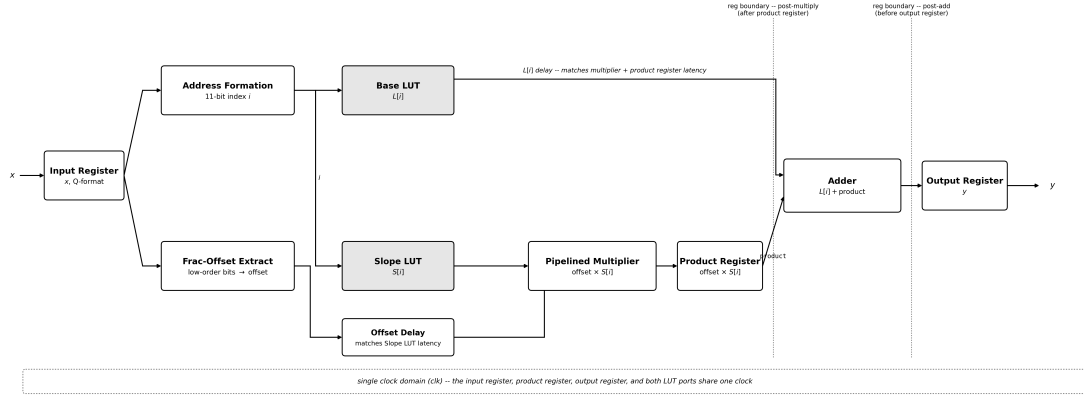


Fig. 2. LUT with Linear Interpolation reciprocal architecture.

computes $y_2 = y_1(2 - xy_1)$, with y_2 used as the final reciprocal estimate. Each iteration consists of two sequential 32×32 fixed-point multiplications with a subtraction from the constant 2 between them, giving four sequential multiplications and two subtraction operations per reciprocal estimate. The datapath is implemented using the deeply pipelined `pipelined_mult32x32_signed` multiplier and the `reciprocal_nr_q8_24` iteration-control structure. Because the four multiplications form a serial dependency chain across the two refinement iterations, the Newton-Raphson architecture has the deepest multiplier pipeline among the evaluated architectures, resulting in a 69-cycle pipeline at the target clock.

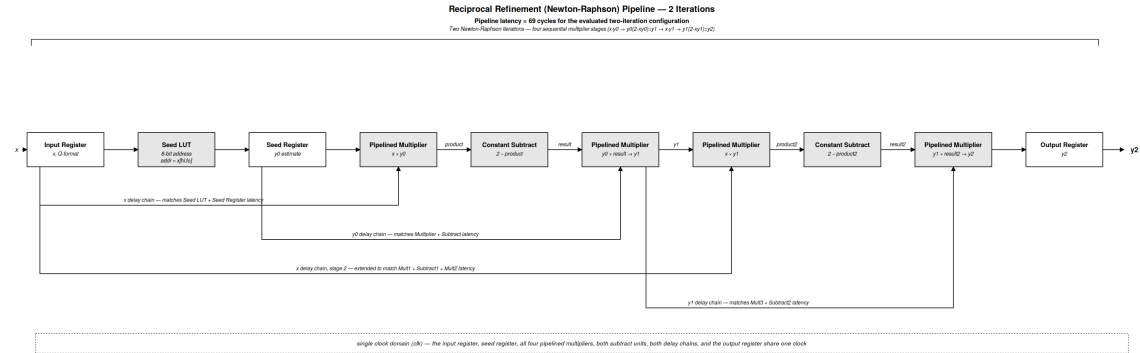


Fig. 3. Newton-Raphson reciprocal architecture.

3.4 Goldschmidt

The Goldschmidt architecture uses the same 8-bit seed table as Newton-Raphson but restructures the refinement as the mutually recursive numerator/denominator update of Eqs. (4)-(5). The two multiplications $F_i \times N_i$ and $F_i \times D_i$ within a given iteration do not depend on one another and can therefore be scheduled in parallel across two multiplier

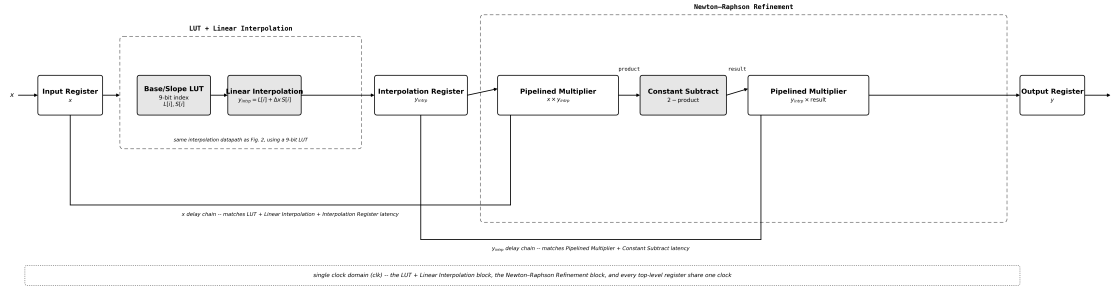


Fig. 5. Hybrid LUT-Interpolation/Newton-Raphson reciprocal architecture.

paths [36]; the DSP utilization figures reported in Table 3 are counted in units of these slices. During implementation, multiple Vivado synthesis and implementation optimization strategies were used, including AggressiveExplore, ExtraTimingOpt, and Explore strategies, together with timing-oriented implementation directives, to obtain the best achievable timing under the stated architectural constraints. The same optimization methodology was applied across the architectures rather than relying on a single default implementation strategy. Two architecture/format combinations—Newton-Raphson and Goldschmidt—additionally required attribute-level intervention during physical synthesis (dont_touch constraints on selected register stages) to prevent Vivado’s retiming optimizer from merging or removing intended pipeline register boundaries; this is discussed further in Section 4.2.

3.7 Algorithmic Complexity

Table 2 summarizes each architecture’s datapath in terms of primitive operation counts—multiplications, additions/subtractions, and memory reads—per complete reciprocal estimate. Direct LUT requires a single memory read and no arithmetic, while LUT-based interpolation adds one multiply and one add to the two table reads used for the stored breakpoint value and slope. Newton-Raphson performs two refinement iterations, with each iteration requiring two sequential multiplications and one subtraction, giving four multiplications and two subtractions per reciprocal estimate. Goldschmidt also performs two refinement iterations; its initialization requires one multiplication to form $D_0 = xy_0$, the first refinement computes $N_1 = N_0 F_0$ and $D_1 = D_0 F_0$ using two parallel multiplications, and the second refinement computes the final $N_2 = N_1 F_1$, giving four multiplications and two subtractions in total. The hybrid architecture combines one interpolation stage with a single Newton-Raphson refinement, resulting in three multiplications and two additions/subtractions per reciprocal estimate. The parallel organization of Goldschmidt reduces sequential multiplier depth relative to Newton-Raphson without reducing the total arithmetic operation count.

4 Experimental Methodology

4.1 Verification Methodology

Functional correctness and numerical accuracy for all five architectures, in both fixed-point formats, were verified against a common double-precision software reference model computing $1/x$ directly. Each architecture was exercised over one million verification vectors swept uniformly across the input domain $x \in [0.5, 2.5]$, with the hardware (or

Table 2. Algorithmic Operation Count per Reciprocal Estimate

Architecture	Mult.	Add/Sub	Mem. Reads	Seed Table Width
Direct LUT	0	0	1	12-bit address
LUT + Interpolation	1	1	2	11-bit address
Newton-Raphson	4	2	1	8-bit address
Goldschmidt	4	2	1	8-bit address
Hybrid	3	2	2	9-bit address

bit-accurate RTL simulation) output compared against the reference value at matching precision. For each output sample, the signed error is defined in least-significant-bit (LSB) units as

$$e[n] = y_{hw}[n] - \text{round}\left(y_{ref}[n] 2^F\right) \quad (9)$$

Here, $y_{hw}[n]$ is the hardware/RTL output represented as a raw integer in the device's Q-format, $y_{ref}[n]$ is the double-precision reference reciprocal, and F is the number of fractional bits for the format under test. For relative-error calculation, the hardware output is converted to real-valued form as $y_{hw,real}[n] = y_{hw}[n] 2^{-F}$. The maximum relative error is then defined as

$$E_{max,rel} = \max_n \left(\frac{|y_{hw,real}[n] - y_{ref}[n]|}{|y_{ref}[n]|} \right) \times 100\% \quad (10)$$

Maximum relative error is used as the primary accuracy metric in the Pareto analysis of Section 6 because, unlike a raw LSB count, it is directly comparable across the two evaluated fixed-point formats: one LSB of Q8.24 (2^{-24}) and one LSB of Q16.16 (2^{-16}) represent different absolute quantities, so an LSB-based error metric cannot be used to rank accuracy across formats without first normalizing by format resolution, which the relative-error metric does implicitly. All pipelined architectures exhibit a warm-up transient during the first N_{cycles} output samples, corresponding to the pipeline fill period before the first valid result reaches the output register; these initial samples are excluded from all summary error statistics and are visualized explicitly in Section 5.3 (pipeline warm-up plots).

4.2 Timing Closure

Timing closure was the most implementation-intensive part of this work, particularly for the two full iterative architectures (Newton-Raphson and Goldschmidt), each of which required multiple revision cycles to close timing at their 387.6 MHz target on the Artix-7 fabric. Several recurring classes of timing violation were encountered and resolved across the iterative architectures' revision history. First, wide fixed-point adders synthesized as single-stage CARRY4 chains repeatedly failed to meet the target period and had to be manually split across pipeline stage boundaries (e.g., separating a cross-term adder from the final high-order sum in the pipelined multiplier). Second, Vivado's register-merge retiming optimization was found to silently undo manually inserted pipeline splits by merging what were intended as two distinct pipeline stages back into one combinational block. Register retiming is, in general, a valuable transformation that moves registers across combinational logic to balance path delays without changing observable circuit behavior [38], and is applied productively to FPGA pipelines in techniques such as C-slow retiming [37]; the difficulty encountered here was that automatic retiming, applied uniformly, merged register boundaries the design deliberately relied on as physical pipeline-stage markers. This was resolved by applying `dont_touch` (and, in one case, a synthesis-level `keep`) attributes to the registers marking the intended stage boundary, since a `shreg_extract="no"` attribute alone was found to be insufficient to prevent this class of retiming. Third, DSP48E1 cascade inference was found to occasionally fuse an

intended register boundary directly into the DSP slice’s internal C-port cascade path, bypassing the pipeline register the designer intended to be a hard stage boundary; this required explicit attribute-level or structural intervention to restore the intended pipeline stage count.

A related correctness (rather than timing) issue surfaced during this process: a one-cycle discrepancy in the reported latency of a signed dedicated-adder submodule (`dsp_add18x18_signed`) caused a systematic, low-probability functional error that was only caught by the full one-million-vector verification sweep described in Section 4.A rather than by smaller directed test sets, underscoring the value of exhaustive-range verification for pipelined fixed-point arithmetic where an off-by-one-cycle latency bug can silently corrupt only a small fraction of the output space. Once identified, the fix consisted of correcting the tracked `MULT_LATENCY` parameter used to align data with its corresponding valid/control signal through the pipeline, together with a corrected operand-port assignment inside the same submodule.

4.3 Power and Energy Measurement

Dynamic and static power figures reported in Section 5.4 are obtained from Vivado’s post-implementation power analysis for each architecture/format combination, using the achieved (post-place-and-route) clock frequency and a representative input-activity profile matched to the verification input distribution described in Section 4.A. Because every evaluated architecture is fully pipelined to produce one valid output per clock cycle once its pipeline is filled, the sustained throughput of each architecture in operations per second is equal to its achieved $F_{\max}$, and the energy consumed per reciprocal operation is computed directly from total power and throughput as

$$E_{op} = \frac{P_{total}}{F_{\max}} \quad (11)$$

This relationship is the basis of every Energy/op figure reported in Table 6 and used throughout the Pareto analysis of Section 6, and was confirmed to reproduce the reported Energy/op values for all ten architecture/format combinations to within measurement rounding. Reported latency in nanoseconds (Table 4) is computed separately from the achieved $F_{\max}$, as the product of the architecture’s fixed pipeline depth in clock cycles and its target clock period:

$$L_{ns} = N_{cycles} \times T_{target} \quad (12)$$

using the target clock period from Table 1 rather than the achieved period, so that latency reflects the architecture’s designed pipeline depth independent of any timing margin achieved beyond the target frequency.

5 Experimental Results

This section presents post-implementation results across resource utilization, timing and latency, numerical accuracy, and power/energy for all five architectures in both evaluated fixed-point formats, followed by a direct cross-format comparison in Section 5.5.

5.1 Resource Utilization

Table 3 reports post-implementation resource utilization for all ten architecture/format combinations. The Direct LUT architecture consumes negligible LUT and flip-flop resources but requires **3.5 BRAMs in Q8.24, tied with LUT-based interpolation for the largest BRAM allocation among the evaluated configurations**, since it must store a full-precision reciprocal value at every one of its 4096 addressable entries. The two full iterative architectures (Newton-Raphson and Goldschmidt) consume an order of magnitude more LUTs and flip-flops than the table-based architectures, reflecting their pipelined multiplier datapaths, and both consume the maximum 20 DSP48E1 slices

allocated to the pipelined multiplier chains. The hybrid architecture sits between the interpolation and full-iterative architectures on LUT/FF count, consistent with its interpolated seed followed by a single Newton-Raphson refinement iteration, while requiring roughly half the DSP count of the full iterative architectures (9 versus 20). This lower DSP utilization results from performing only one Newton-Raphson refinement iteration after interpolation, rather than the two refinement iterations used by the standalone Newton-Raphson architecture.

Table 3. Post-Implementation Resource Utilization

Architecture	Format	LUT	LUT%	FF	DSP	BRAM
Direct LUT	Q8.24	19	0.03%	84	0	3.5
Direct LUT	Q16.16	2	0.003%	51	0	2.5
LUT + Interp.	Q8.24	159	0.25%	685	1	3.5
LUT + Interp.	Q16.16	119	0.19%	647	1	2.0
Newton-Raphson	Q8.24	966	1.52%	3612	20	0.5
Newton-Raphson	Q16.16	849	1.34%	3340	20	0.5
Goldschmidt	Q8.24	846	1.33%	3656	20	0.5
Goldschmidt	Q16.16	722	1.14%	3385	20	0.5
Hybrid	Q8.24	768	1.21%	2626	9	1.0
Hybrid	Q16.16	799	1.26%	2586	9	1.0

5.2 Timing and Latency

Table 4 reports the achieved maximum frequency ($F_{\max}$), worst negative slack (WNS) at the target clock period, pipeline depth, and corresponding latency calculated using Eq. (12). All ten architecture/format combinations achieve timing closure with positive slack at their respective target periods. The Direct LUT architecture has the shortest pipeline depth, requiring 5 cycles for Q8.24 and 3 cycles for Q16.16, resulting in the lowest absolute latency among the evaluated architectures. In contrast, the Newton-Raphson architecture requires the deepest pipeline at 69 cycles, reflecting the four sequentially dependent multiplication stages arising from its two refinement iterations, as described in Section 3.C. Goldschmidt reduces the pipeline depth to 54 cycles by exploiting parallelism between intermediate multiplier operations, representing an approximately 22% reduction in pipeline depth relative to Newton-Raphson.

Table 4. Post-Implementation Timing and Latency

Architecture	Format	$F_{\max}$ (MHz)	WNS (ns)	Latency (cyc.)	Latency (ns)	Throughput (Mops/s)
Direct LUT	Q8.24	392.77	0.034	5	12.90	392.77
Direct LUT	Q16.16	519.21	0.654	3	7.74	519.21
LUT + Interp.	Q8.24	429.74	0.253	16	41.28	429.74
LUT + Interp.	Q16.16	424.45	0.224	16	41.28	424.45
Newton-Raphson	Q8.24	399.20	0.075	69	178.02	399.20
Newton-Raphson	Q16.16	392.62	0.033	69	178.02	392.62
Goldschmidt	Q8.24	396.35	0.057	54	139.32	396.35
Goldschmidt	Q16.16	394.94	0.048	54	139.32	394.94
Hybrid	Q8.24	316.06	0.169	40	133.20	316.06
Hybrid	Q16.16	305.06	0.055	40	133.20	305.06

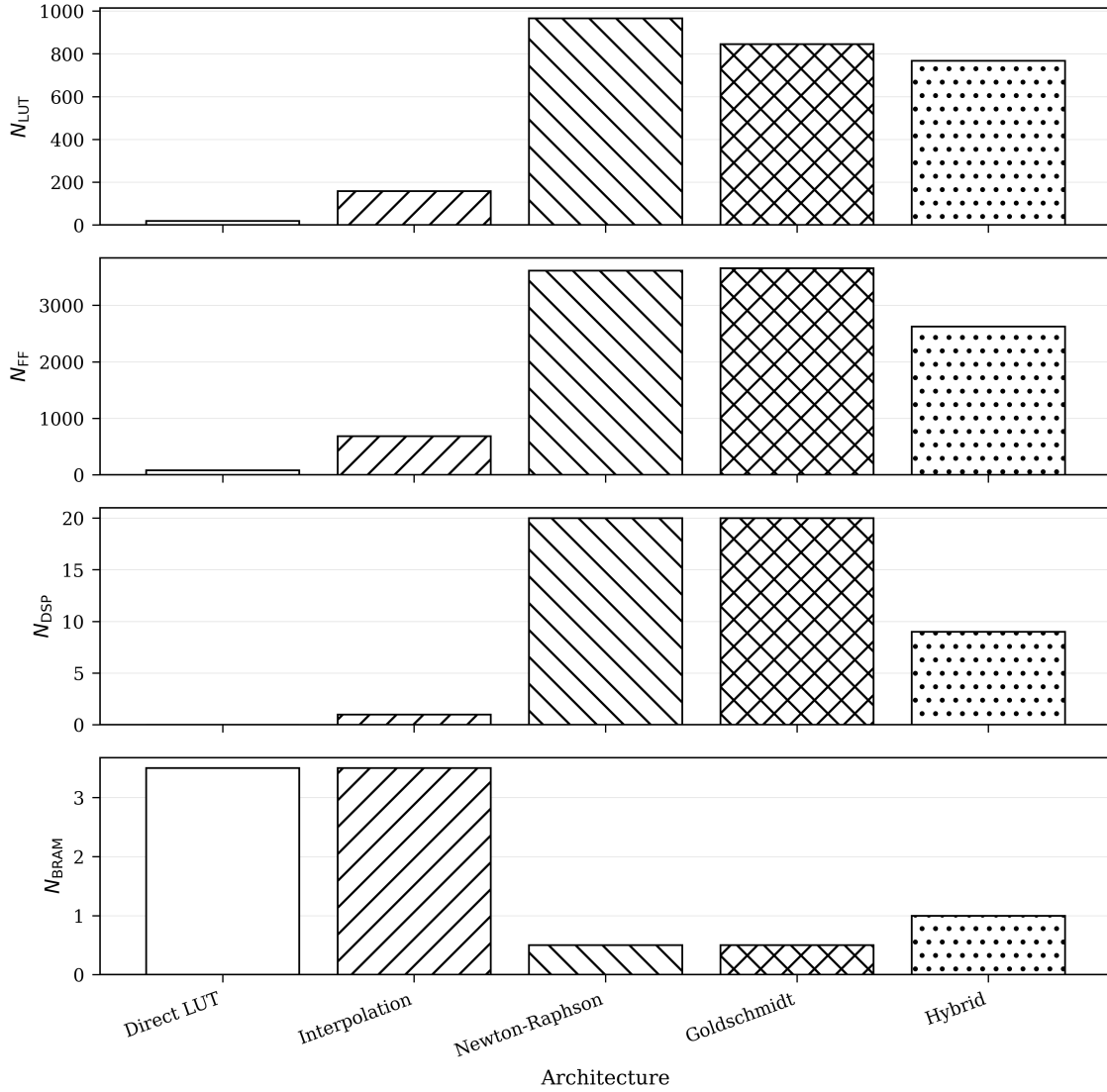


Fig. 6. Post-implementation LUT, flip-flop, DSP, and BRAM utilization by architecture, Q8.24.

5.3 Numerical Accuracy

Table 5 summarizes the four accuracy metrics defined in Section 4.A for all ten architecture/format combinations. As expected from the architectural discussion in Sections 3.A-3.D, the two table-based architectures exhibit the largest maximum relative error (0.098% for Direct LUT and 0.0104% for LUT with Interpolation, both in Q8.24), while the two full iterative architectures achieve error more than three orders of magnitude smaller ($1.5 \times 10^{-5}\%$ for Newton-Raphson and $2.0 \times 10^{-5}\%$ for Goldschmidt, Q8.24). The hybrid architecture, benefiting from a single refinement iteration applied to an already-interpolated seed, achieves a maximum relative error of $4.09 \times 10^{-4}\%$ —roughly 25× worse than the full

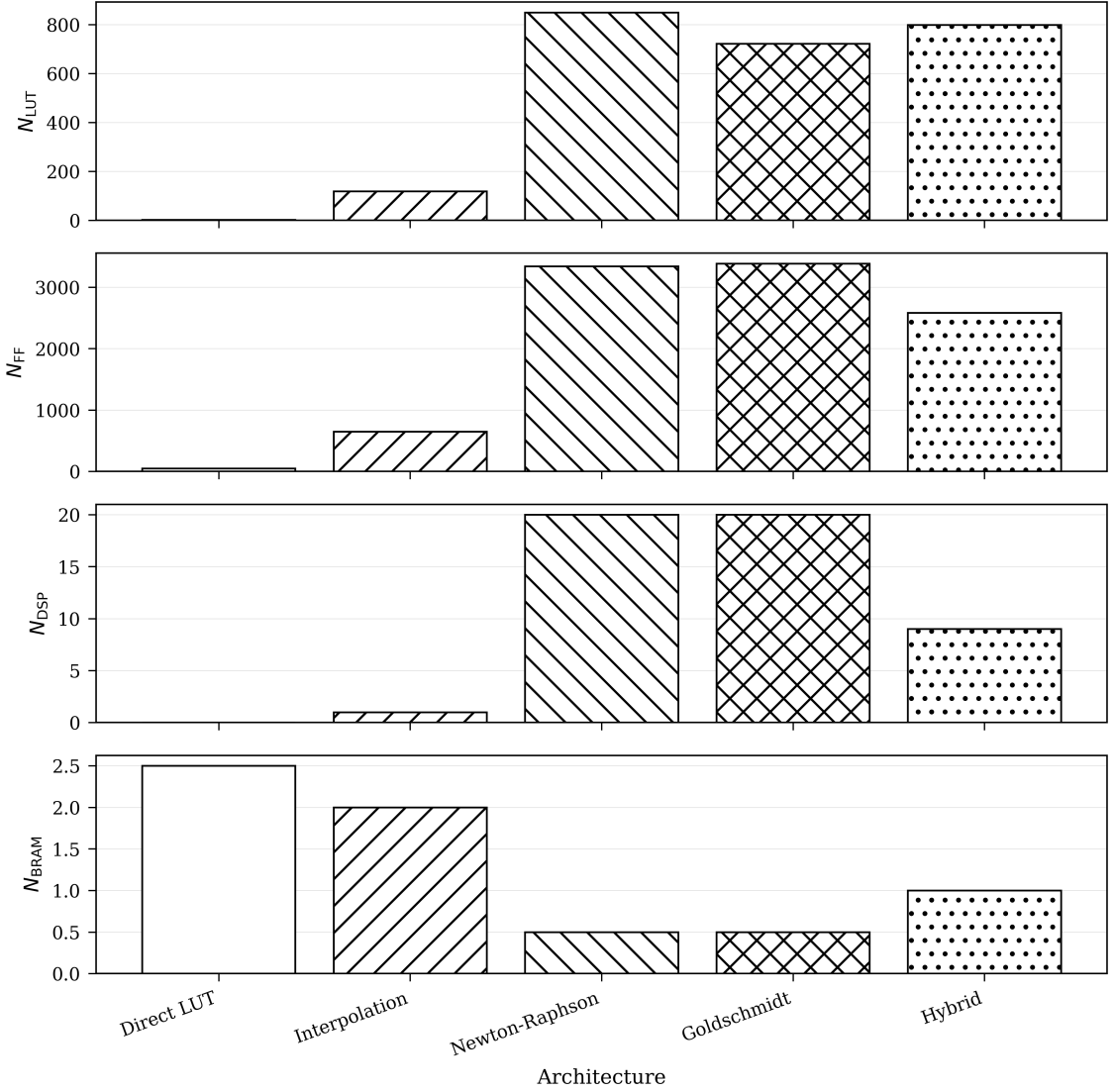


Fig. 7. Post-implementation LUT, flip-flop, DSP, and BRAM utilization by architecture, Q16.16.

iterative architectures but more than 25× better than interpolation alone, consistent with the intermediate positioning described in Section 3.E.

Figs. 10-11 show the distribution of signed error across all five Q8.24 architectures, first in raw LSB units and then on a log-compressed $|e|$ scale to make the tails of the heavier-tailed distributions (Direct LUT, Hybrid) visible alongside the tightly concentrated iterative architectures. The corresponding Q16.16 distributions are shown in Figs. 16-17.

Fig. 12 decomposes the error population for each architecture into the fraction of outputs with zero LSB error relative to the quantized fixed-point reference and the distribution of nonzero errors. The zero-LSB fraction varies substantially

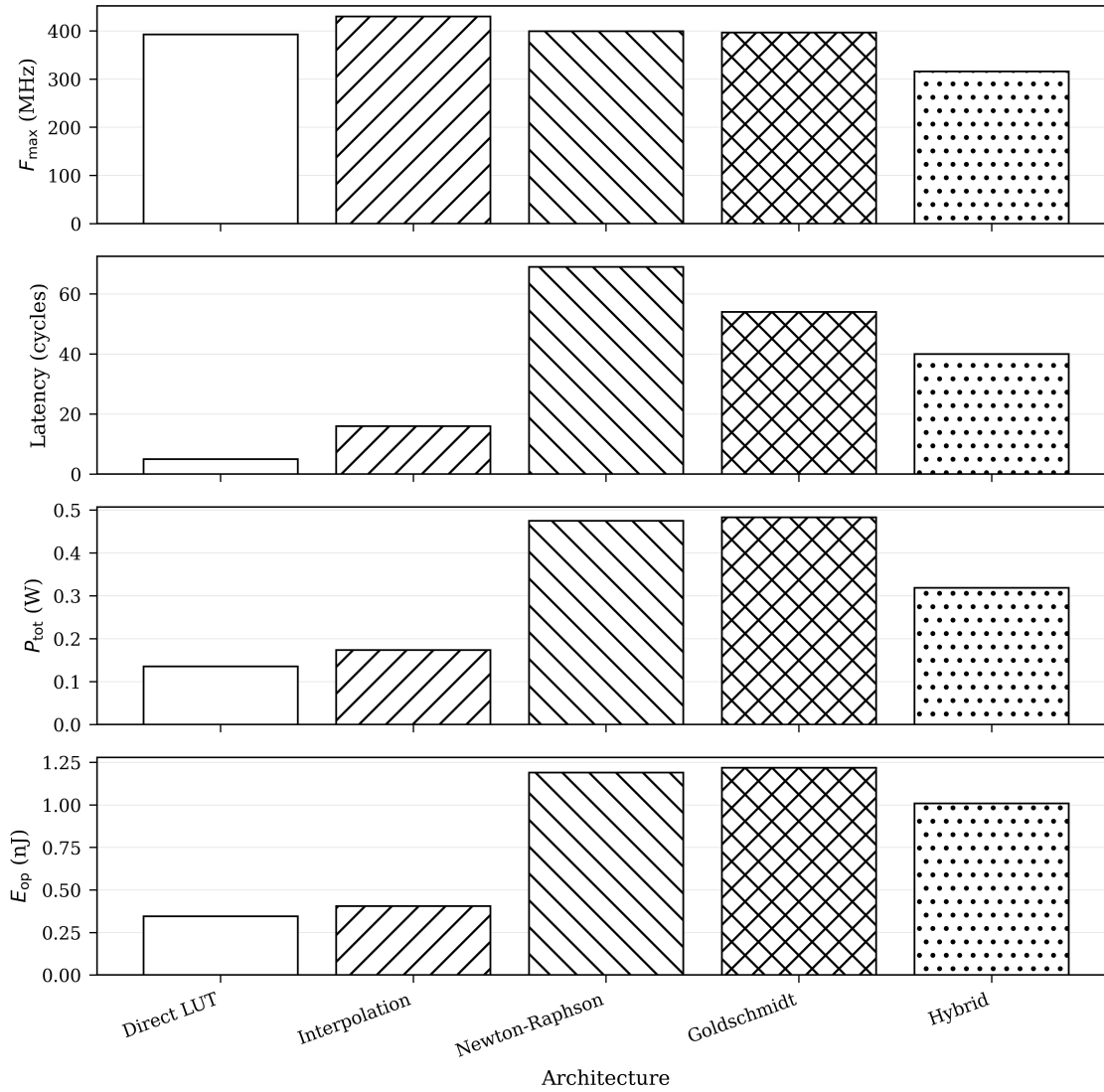


Fig. 8. Post-implementation $F_{\max}$, latency (cycles), total power, and energy per operation by architecture, Q8.24.

across the architectures. Newton-Raphson and Goldschmidt produce exact agreement with the quantized fixed-point reference for a large fraction of test vectors, whereas the hybrid architecture exhibits predominantly nonzero errors despite its substantially lower maximum relative error than the interpolation-only design.

Figs. ??-13 show how the maximum absolute and maximum relative error vary as a function of the input value x across the domain $[0.5, 2.5]$. For all five architectures, error is concentrated toward the lower end of the input domain, where the local slope of $1/x$ is steepest and a fixed quantization step in x therefore maps to a larger quantization

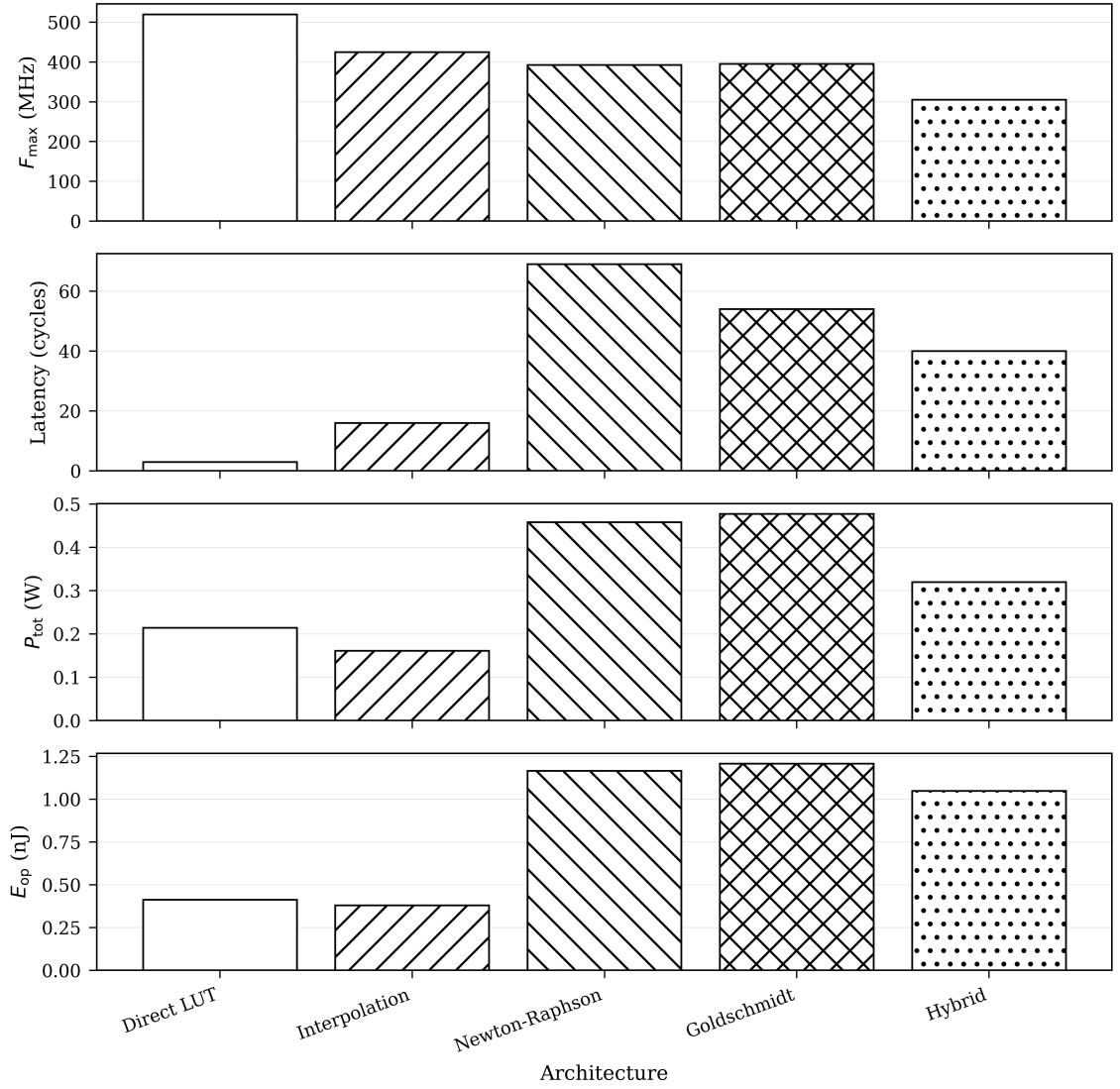


Fig. 9. Post-implementation $F_{\max}$, latency (cycles), total power, and energy per operation by architecture, Q16.16.

step in $1/x$; error decreases monotonically (Direct LUT) or in a sawtooth pattern tied to breakpoint/table boundaries (Interpolation, Goldschmidt, Hybrid) as x increases toward 2.5.

Fig. 14 shows the pipeline warm-up transient discussed in Section 4.A: for each architecture, the first N_{cycles} output samples (shaded) correspond to the pipeline fill period and do not represent valid steady-state outputs; these samples are excluded from all summary statistics reported in Table 5. Fig. 15 presents the complementary cumulative view, showing absolute error as a function of output percentile across the full one-million-vector sweep, which makes clear

Table 5. Numerical Accuracy Summary (1,000,000-Vector Sweep, $x \in [0.5, 2.5]$)

Architecture	Format	Max Abs Error (real)	MAE (real)	RMSE (LSB)	Max Rel. Error (%)
Direct LUT	Q8.24	1.950×10^{-3}	1.95×10^{-4}	5440.20	0.097596
Direct LUT	Q16.16	1.892×10^{-3}	1.89×10^{-4}	20.75	0.094787
LUT + Interpolation	Q8.24	2.079×10^{-4}	≈ 0	11.70	0.010396
LUT + Interpolation	Q16.16	1.831×10^{-4}	7.49×10^{-6}	0.71	0.009156
Newton-Raphson	Q8.24	1.192×10^{-7}	≈ 0	0.45	0.000015
Newton-Raphson	Q16.16	3.526×10^{-5}	3.11×10^{-6}	0.45	0.003815
Goldschmidt	Q8.24	1.788×10^{-7}	2.15×10^{-8}	0.62	0.000020
Goldschmidt	Q16.16	3.052×10^{-5}	3.81×10^{-6}	0.51	0.003814
Hybrid	Q8.24	8.166×10^{-6}	1.60×10^{-6}	38.60	0.000409
Hybrid	Q16.16	7.629×10^{-5}	3.86×10^{-6}	0.59	0.004118

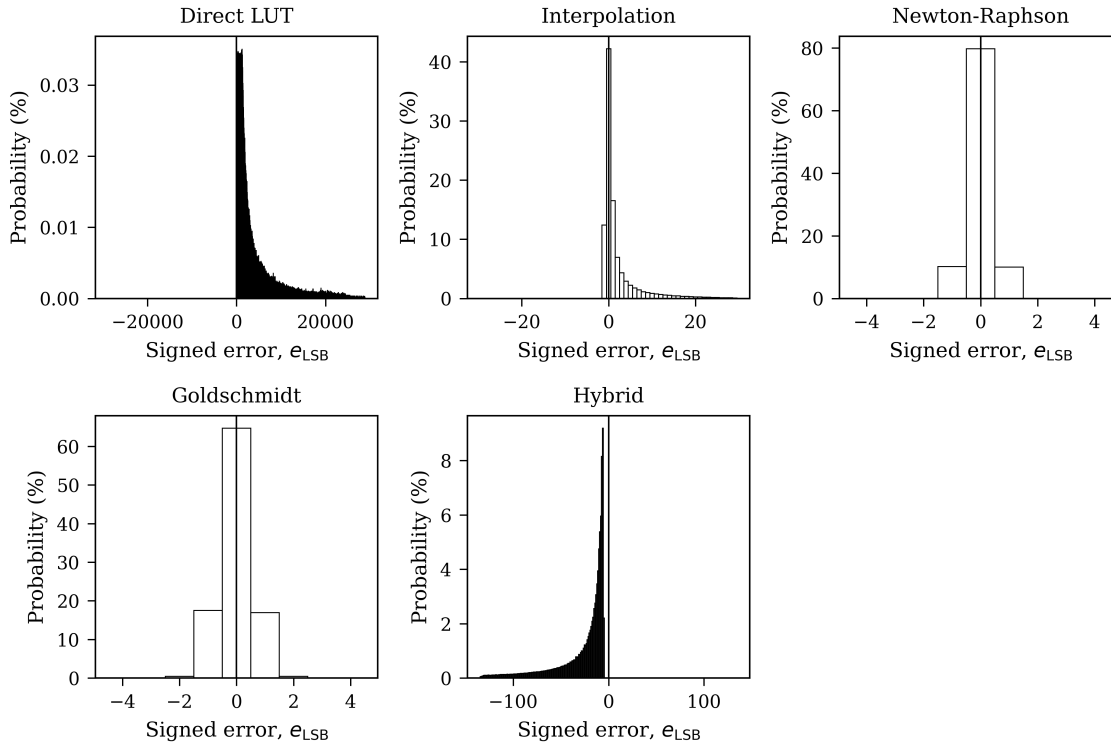


Fig. 10. Signed error distributions by architecture, Q8.24 (raw LSB units).

that all five architectures' worst-case error is concentrated in a small percentile tail rather than being representative of typical-case behavior.

The corresponding Q16.16 error-distribution, worst-case, warm-up, and percentile figures are reproduced in Figs. 16-21 for direct comparison against the Q8.24 results above; the cross-format differences visible between these figure pairs are discussed quantitatively in Section 5.5.

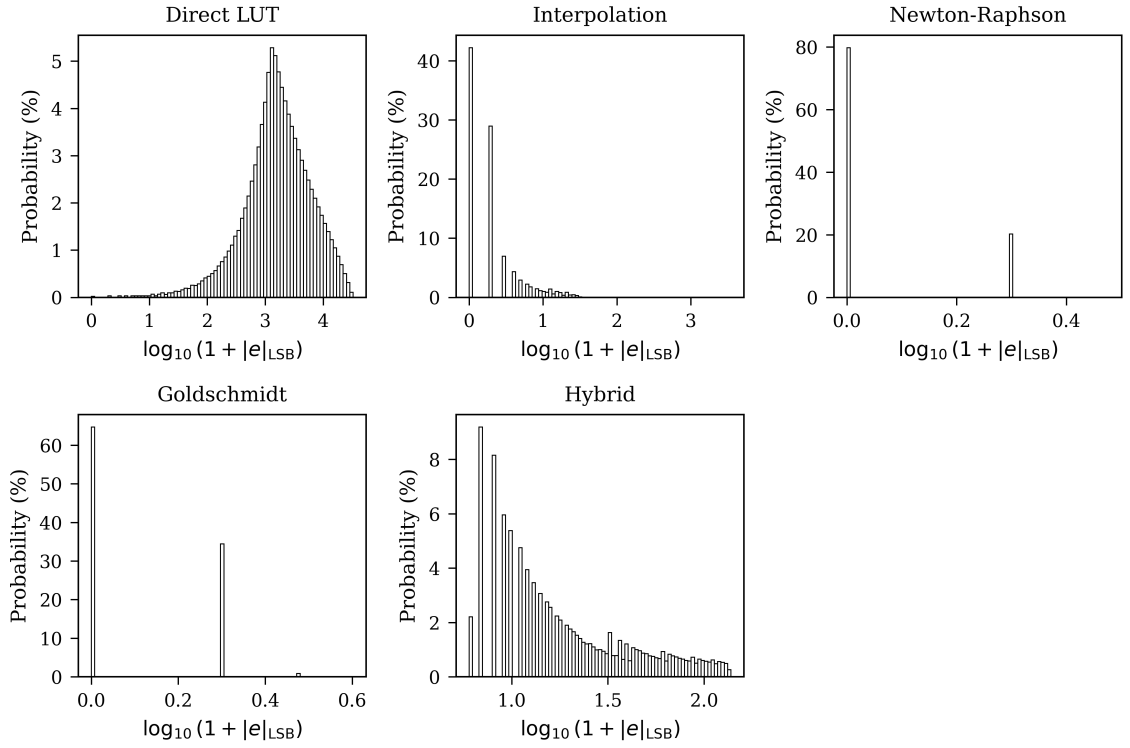


Fig. 11. Absolute error distributions by architecture, Q8.24 ($\log_{10}(1 + |e|)$ scale).

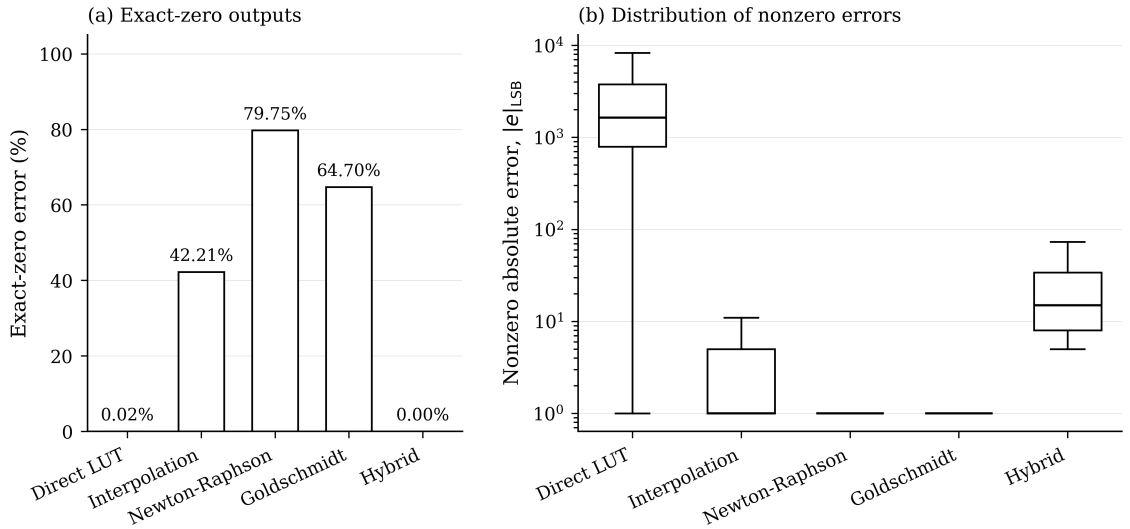
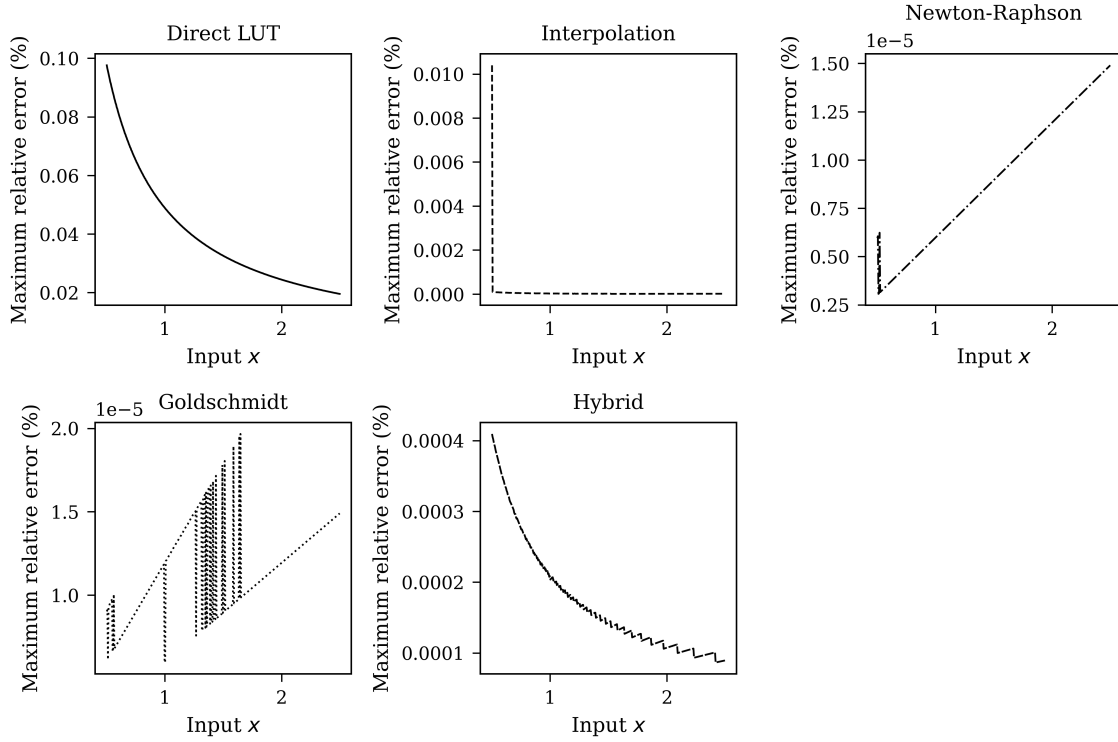


Fig. 12. Zero-LSB output fraction and distribution of nonzero errors by architecture, Q8.24.

Fig. 13. Maximum relative error as a function of input value x , by architecture, Q8.24.

5.4 Power and Energy

Table 6 reports dynamic, static, and total power together with the derived energy per operation (Eq. (11)) for all ten architecture/format combinations. Total power scales closely with DSP and flip-flop count: the two full iterative architectures consume the most total power (0.475-0.483 W) and, combined with their long pipeline latency, the most energy per operation (1.19-1.22 nJ/op) of any evaluated architecture. The Direct LUT architecture is the most energy-efficient in both formats (0.346-0.412 nJ/op), reflecting both its minimal dynamic power draw and its short pipeline. Static power is essentially constant (0.091-0.092 W) across all ten combinations, as expected since it is dominated by device leakage rather than architecture-specific switching activity.

5.5 Cross-Format Comparison

Figs. 22-25 directly compare Q8.24 and Q16.16 across achieved $F_{\max}$ and latency, LUT/DSP resource usage, total power and energy per operation, and full four-metric resource utilization, respectively. Two cross-format effects are evident. First, Direct LUT and LUT-with-Interpolation achieve markedly higher $F_{\max}$ in Q16.16 than in Q8.24, with Direct LUT increasing from 392.8 MHz to 519.2 MHz. This difference is consistent with the narrower output representation and the resulting implementation of the BRAM/output datapath, although the exact timing improvement depends on the post-implementation critical paths. In contrast, Newton-Raphson, Goldschmidt, and Hybrid show comparatively smaller

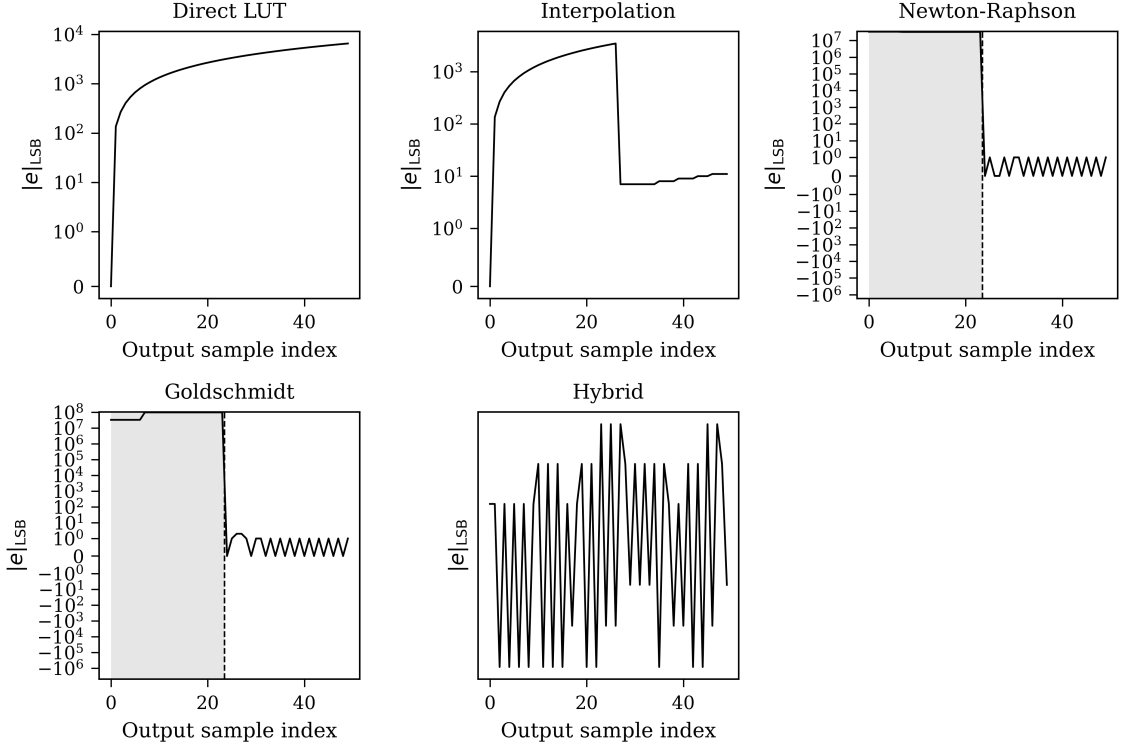


Fig. 14. Pipeline warm-up transient by architecture (shaded region = pipeline fill period), Q8.24.

Table 6. Post-Implementation Power and Energy

Architecture	Format	Dynamic P (W)	Static P (W)	Total P (W)	Energy/op (nJ)
Direct LUT	Q8.24	0.045	0.091	0.136	0.346
Direct LUT	Q16.16	0.122	0.092	0.214	0.412
LUT + Interp.	Q8.24	0.083	0.091	0.174	0.405
LUT + Interp.	Q16.16	0.070	0.091	0.161	0.379
Newton-Raphson	Q8.24	0.383	0.092	0.475	1.190
Newton-Raphson	Q16.16	0.366	0.092	0.458	1.167
Goldschmidt	Q8.24	0.391	0.092	0.483	1.219
Goldschmidt	Q16.16	0.385	0.092	0.477	1.208
Hybrid	Q8.24	0.227	0.092	0.319	1.009
Hybrid	Q16.16	0.228	0.092	0.320	1.049

$F_{\max}$ differences between the two formats, reflecting the greater influence of their pipelined arithmetic datapaths on the achieved timing.

Second, the iterative and hybrid architectures exhibit substantially different accuracy behavior across the two formats. Newton-Raphson and Goldschmidt, both seeded from 8-bit coarse lookup tables, show substantially higher maximum relative error in Q16.16 than in Q8.24, by approximately two orders of magnitude. The Hybrid architecture, which

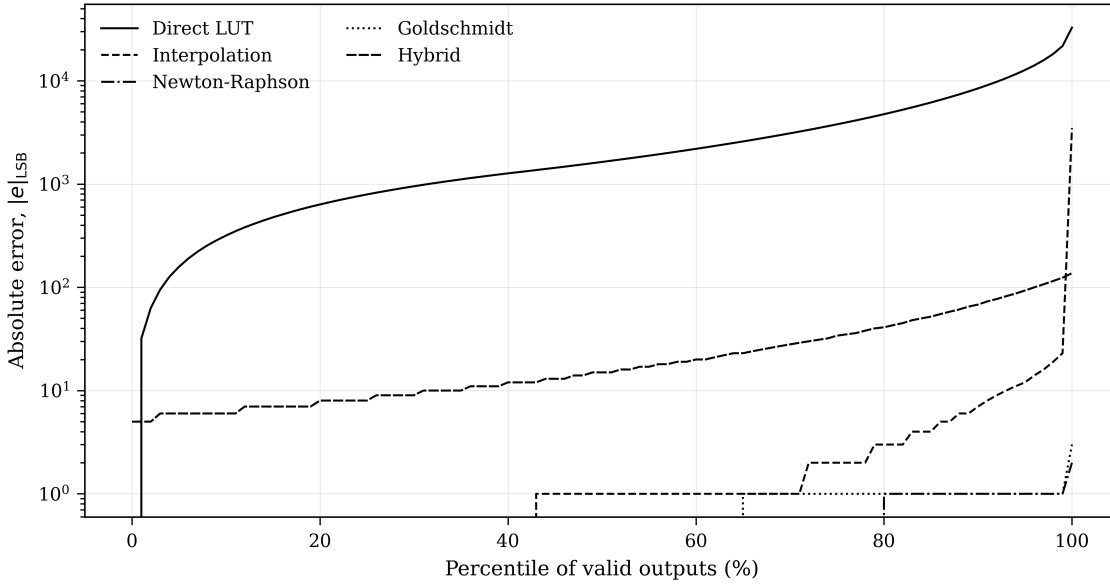


Fig. 15. Absolute error as a function of output percentile, all architectures, Q8.24.

uses a 9-bit interpolated seed followed by a Newton-Raphson refinement, also shows a higher maximum relative error in Q16.16. This behavior is consistent with the reduced fractional precision of Q16.16, which provides fewer bits for representing intermediate and final correction terms throughout the fixed-point refinement process.

These results illustrate that the effect of fixed-point format extends beyond numerical representation alone and influences the measured timing and accuracy characteristics of the architectures. Consequently, the relative trade-offs among accuracy, latency, and energy depend on both the reciprocal algorithm and the fixed-point format used by the surrounding system.

6 Pareto Trade-off Analysis

6.1 Pareto Formulation

The results of Section 5 show that no single architecture dominates the others across every metric of interest. The relative ordering depends on the fixed-point format and the metric considered: the iterative architectures provide substantially lower numerical error than the table-based approaches, but generally incur higher latency and energy per operation, whereas the Direct LUT architecture provides the shortest latency with low energy per operation at the cost of higher approximation error. To characterize these trade-offs rigorously rather than anecdotally, this section applies a three-objective Pareto analysis over maximum relative error, latency, and energy per operation:

$$\text{minimize } \{E_{\max,rel}, L_{ns}, E_{op}\} \quad (13)$$

Hardware resource utilization (LUT, FF, DSP, BRAM) is deliberately reported separately (Table 3) rather than folded into the Pareto objective set above. A composite resource metric would require assigning relative weights across four resource types with different device-level scarcity and cost characteristics (e.g., BRAM versus DSP), and any such

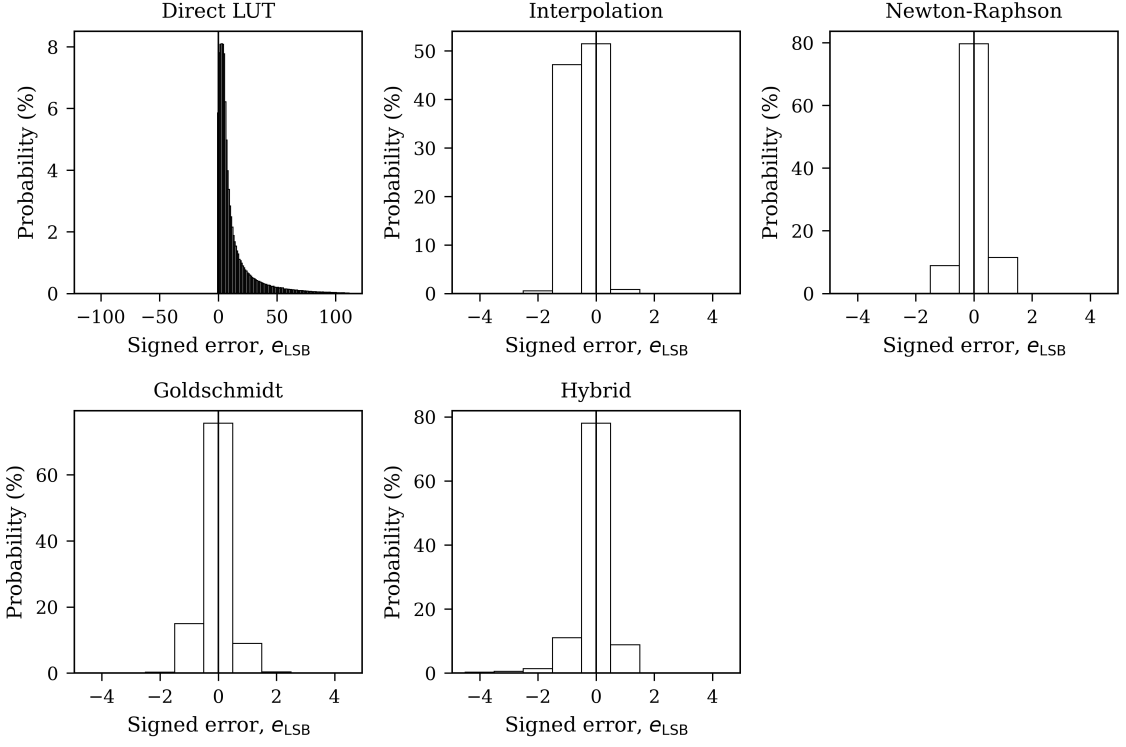


Fig. 16. Signed error distributions by architecture, Q16.16 (raw LSB units).

weighting would introduce a design choice into what is intended to be an objective, data-driven comparison; a design's resource cost is instead read directly from Table 3 alongside its position on the Pareto front. Given this three-objective formulation, an architecture A is said to dominate an architecture B if A is no worse than B on all three objectives and strictly better on at least one:

$$A \succ B \iff (\forall k \in \{E_{rel}, L, E_{op}\} : A_k \leq B_k) \wedge (\exists k : A_k < B_k) \quad (14)$$

An architecture is Pareto-optimal (non-dominated) if no other evaluated architecture dominates it under this relation. Energy-delay product ($EDP = E_{op} \times L_{ns}$) is computed separately as a secondary, derived metric (Section 6.D) rather than substituted for latency and energy as a combined objective, since two architectures with identical EDP can have substantially different individual energy and latency characteristics that matter for system-level integration (e.g., a hard real-time latency budget versus a battery energy budget), and collapsing the two into one number would discard that distinction.

6.2 Pareto Fronts

Fig. 26 plots each architecture's position in the latency-energy plane for both fixed-point formats, with marker size inversely related to maximum relative error: larger markers indicate lower maximum relative error. A single global marker-size scale is shared across both panels so that marker sizes are directly comparable between the Q8.24 and Q16.16

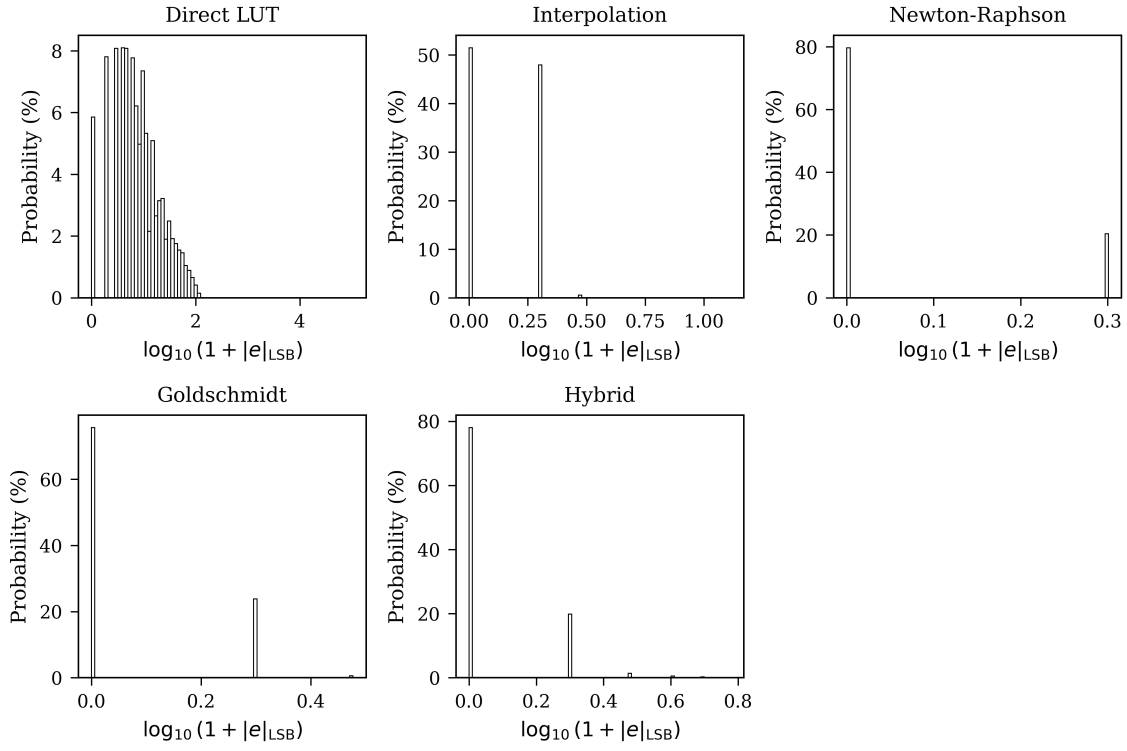


Fig. 17. Absolute error distributions by architecture, Q16.16 ($\log_{10}(1 + |e|)$ scale).

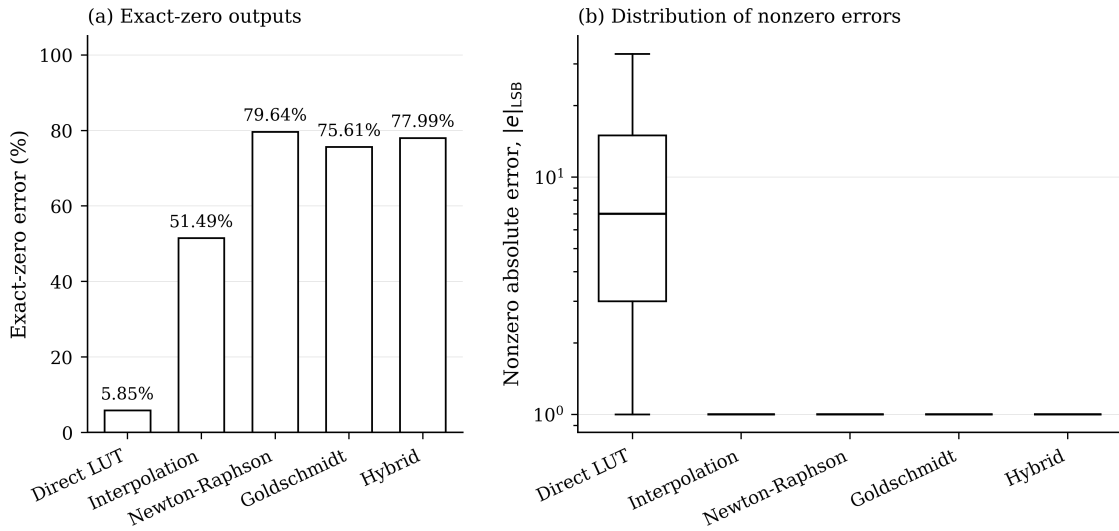


Fig. 18. Zero-LSB output fraction and distribution of nonzero errors by architecture, Q16.16.

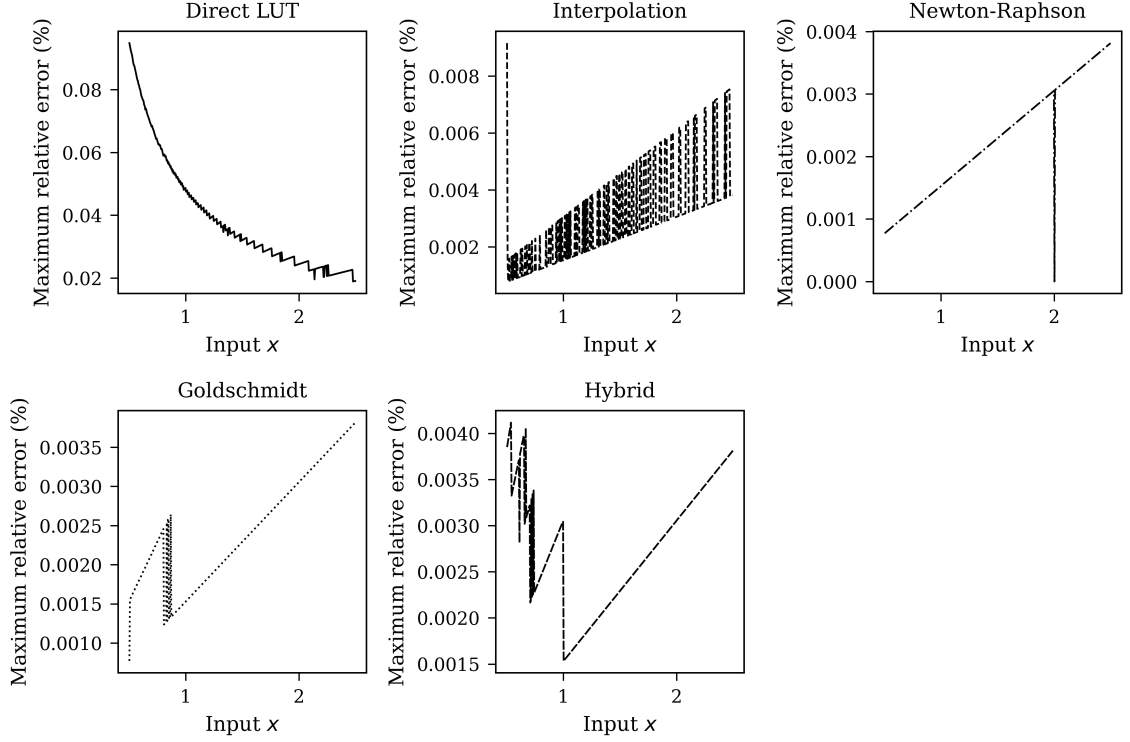


Fig. 19. Maximum relative error as a function of input value x , by architecture, Q16.16.

results rather than only within each format. Applying the dominance relation of Eq. (14) to all five architectures within each format shows that all five architectures are non-dominated in both Q8.24 and Q16.16: no evaluated architecture is simultaneously no worse than another architecture on all three objectives while being strictly better on at least one. Thus, each architecture represents a distinct trade-off point in the three-dimensional accuracy-latency-energy objective space, and the resulting Pareto set reflects different combinations of numerical accuracy, latency, and energy per operation rather than a single architecture dominating across all three objectives.

6.3 Trade-off Analysis

Within the Q8.24 non-dominated set, Direct LUT occupies the low-latency and low-energy region of the trade-off surface, with a latency of 12.9 ns, energy of 0.346 nJ/op, and maximum relative error of 0.098%. Newton-Raphson occupies the high-accuracy region, achieving $1.5 \times 10^{-5}\%$ maximum relative error at 178.0 ns latency and 1.190 nJ/op. Goldschmidt provides comparable numerical accuracy, with a maximum relative error of $2.0 \times 10^{-5}\%$, while reducing latency by approximately 22% to 139.3 ns. This latency reduction is accompanied by a small increase in energy per operation, from 1.190 to 1.219 nJ/op (approximately 2.4%). Thus, the measured Goldschmidt implementation represents a lower-latency alternative to Newton-Raphson when latency is prioritized over a small difference in energy per operation.

The hybrid architecture occupies an intermediate region of the trade-off surface. It achieves 133.2 ns latency and 1.009 nJ/op, both lower than the corresponding values of the full iterative architectures, while achieving a maximum

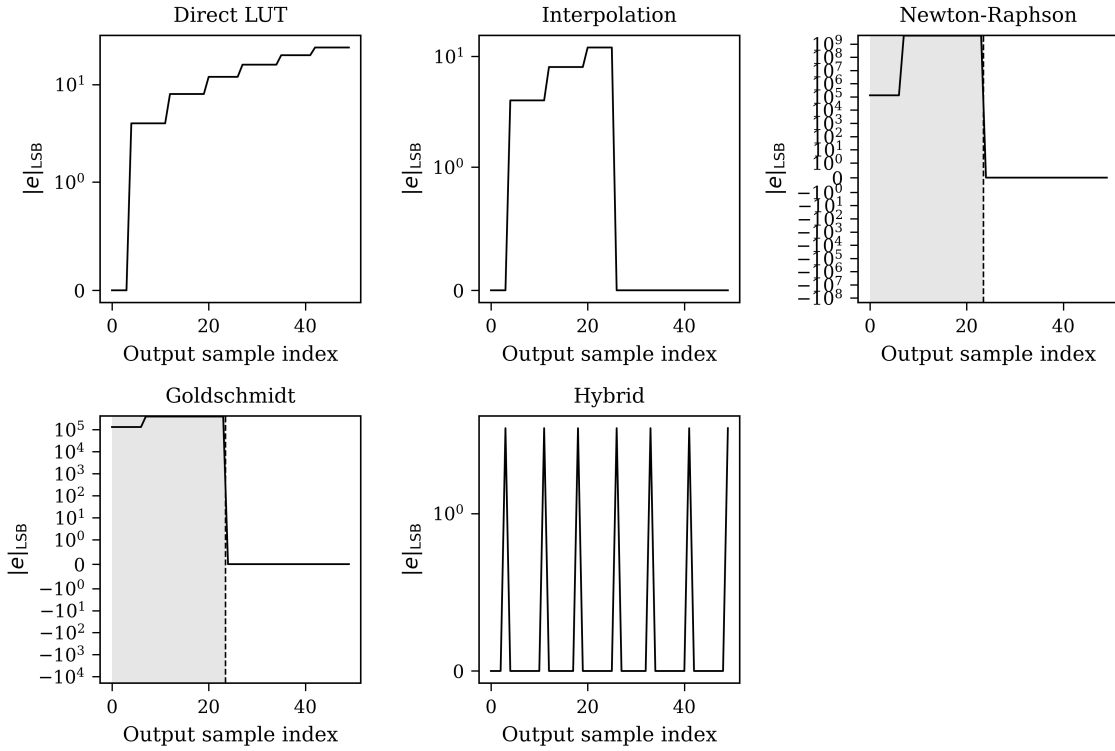


Fig. 20. Pipeline warm-up transient by architecture (shaded region = pipeline fill period), Q16.16.

relative error of $4.09 \times 10^{-4}\%$. This represents more than an order-of-magnitude improvement over interpolation-only accuracy ($1.04 \times 10^{-2}\%$). The result demonstrates the distinct trade-off obtained by combining a 9-bit interpolated seed with a single Newton-Raphson refinement, providing an intermediate accuracy, latency, energy, and DSP-utilization point between table-based and fully iterative approaches.

6.4 Energy-Delay Product

Table 7 reports energy-delay product ($EDP = E_{op} \times L_{ns}$) for all ten architecture/format combinations as a secondary, derived summary metric, per the reasoning in Section 6.A. EDP reproduces the same broad ranking as the individual latency and energy metrics—Direct LUT lowest, the full iterative architectures highest—but is useful as a single figure of merit when comparing architectures for applications where the product of speed and efficiency, rather than either quantity alone, is the relevant system-level cost function (for example, when energy is consumed only while a result is pending, so a design that is twice as fast for the same power is preferable to a design with half the power at the same latency).

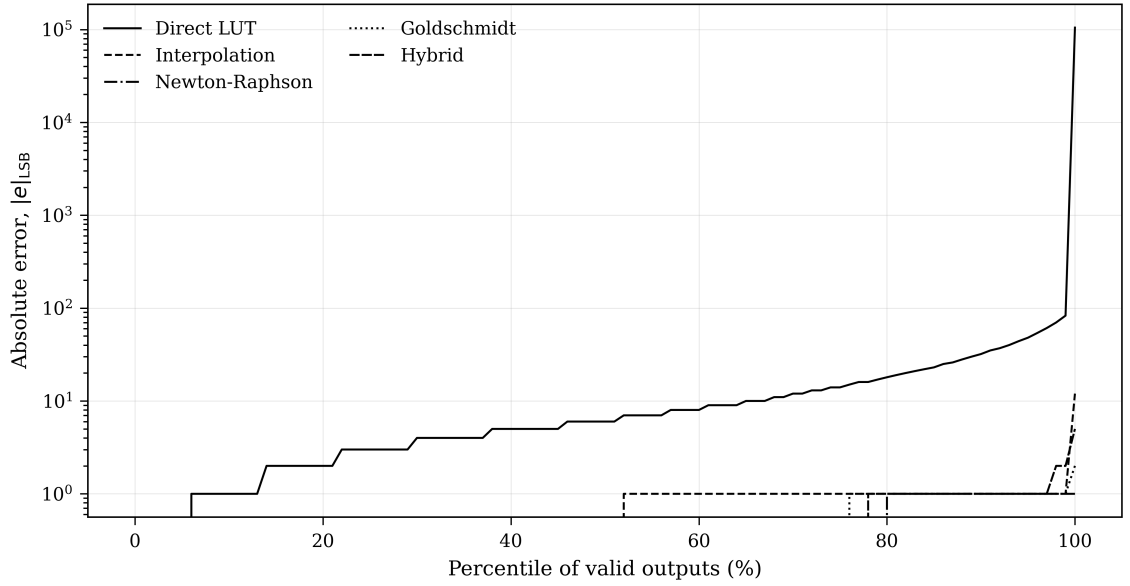


Fig. 21. Absolute error as a function of output percentile, all architectures, Q16.16.

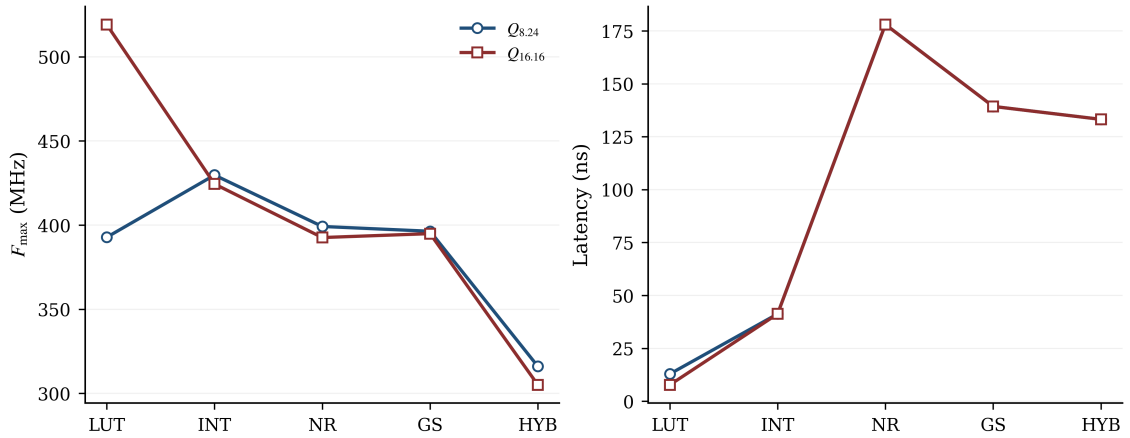


Fig. 22. Achieved $F_{\max}$ and latency (ns) by architecture, Q8.24 vs. Q16.16.

7 Discussion

7.1 Architecture-Level Trade-offs

The results of Sections 5 and 6 illustrate several distinct architecture-level trade-offs. For applications in which latency and energy are important constraints and an error on the order of **0.1%** is acceptable, the **Direct LUT** architecture provides the shortest pipeline and negligible logic utilization, while requiring one of the largest BRAM footprints among the evaluated designs. For the **Q8.24** results, applications requiring maximum relative error below approximately

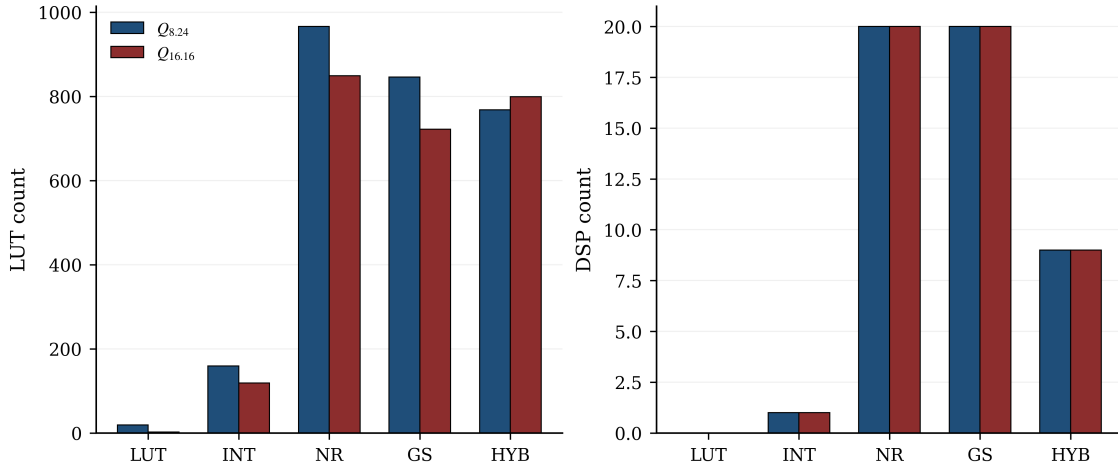


Fig. 23. LUT and DSP utilization by architecture, Q8.24 vs. Q16.16.

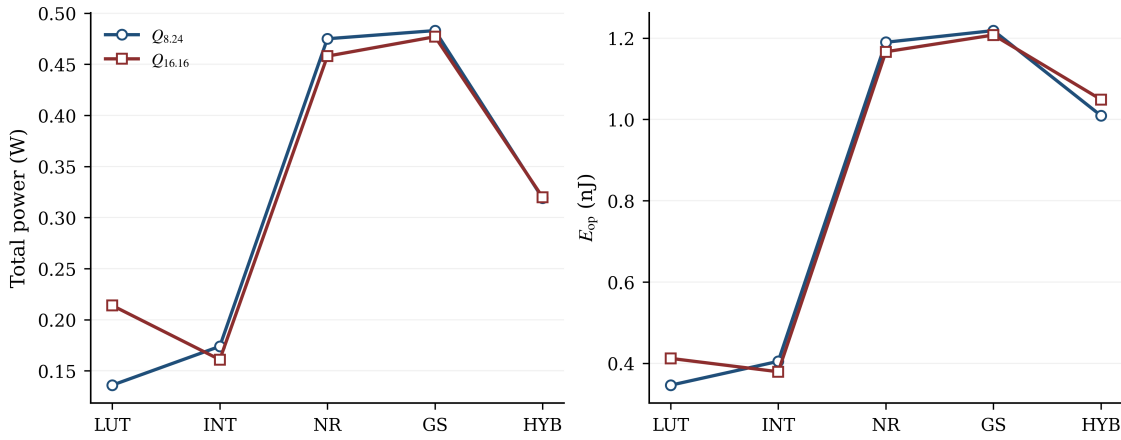


Fig. 24. Total power and energy per operation by architecture, Q8.24 vs. Q16.16.

$10^{-4}\%$ are served by the full iterative architectures, which provide substantially higher numerical accuracy than the table-based approaches. Within this accuracy range, **Goldschmidt** achieves comparable accuracy to Newton-Raphson while reducing latency by approximately 22%. This latency reduction is accompanied by an energy-per-operation increase of approximately 2.4% in Q8.24 and 3.5% in Q16.16. The **hybrid** architecture provides a distinct intermediate trade-off by combining a 9-bit interpolated seed with a single Newton-Raphson refinement. It achieves substantially lower error than interpolation alone while reducing the hardware cost of full iterative refinement, with approximately 75% of the Newton-Raphson latency and 45% of the DSP utilization of either full iterative architecture.

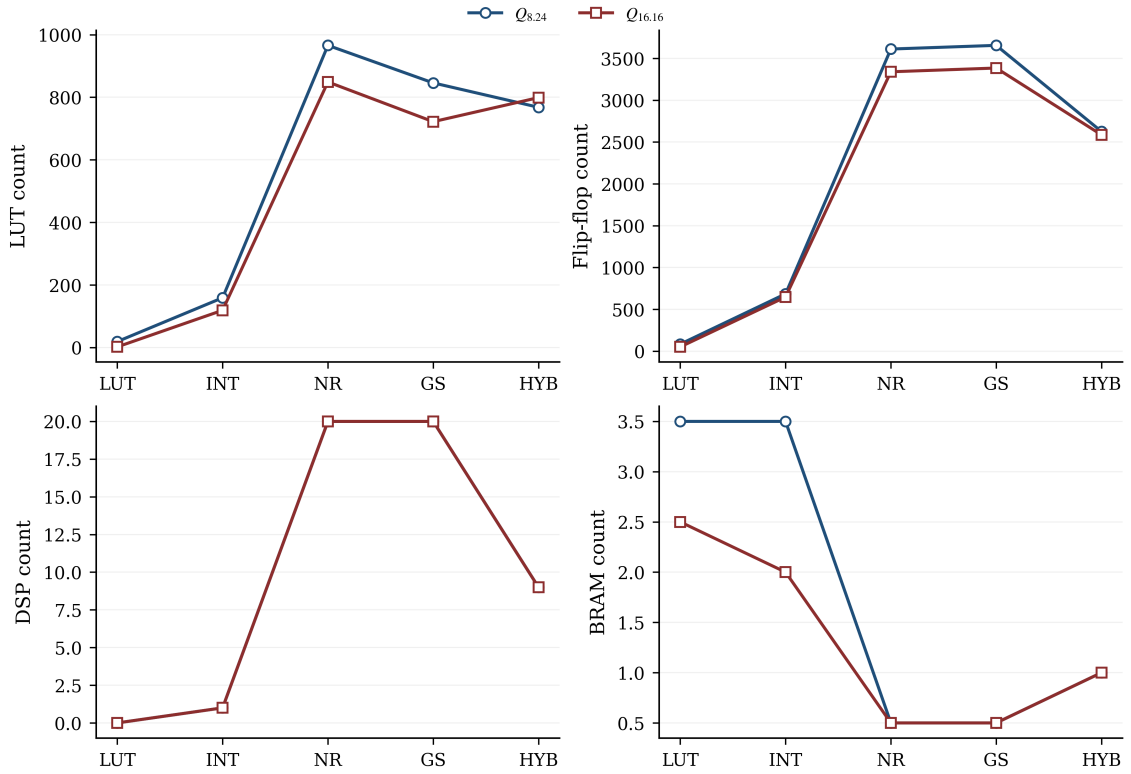


Fig. 25. Full resource utilization (LUT, FF, DSP, BRAM) by architecture, Q8.24 vs. Q16.16.

Table 7. Energy-Delay Product (EDP = Energy/op $\times$ Latency)

Architecture	Format	Energy/op (nJ)	EDP (nJ-ns)
Direct LUT	Q8.24	0.346	4.46
Direct LUT	Q16.16	0.412	3.19
LUT + Interpolation	Q8.24	0.405	16.72
LUT + Interpolation	Q16.16	0.379	15.66
Newton-Raphson	Q8.24	1.190	211.80
Newton-Raphson	Q16.16	1.167	207.68
Goldschmidt	Q8.24	1.219	169.79
Goldschmidt	Q16.16	1.208	168.24
Hybrid	Q8.24	1.009	134.44
Hybrid	Q16.16	1.049	139.73

7.2 Applicability

The reciprocal approximation architectures evaluated in this work are applicable wherever a fixed-point, division-free reciprocal is required as a building block within a larger FPGA arithmetic pipeline. Representative applications include normalization stages in adaptive and statistical signal processing, such as computing a z-score or an Ornstein-Uhlenbeck

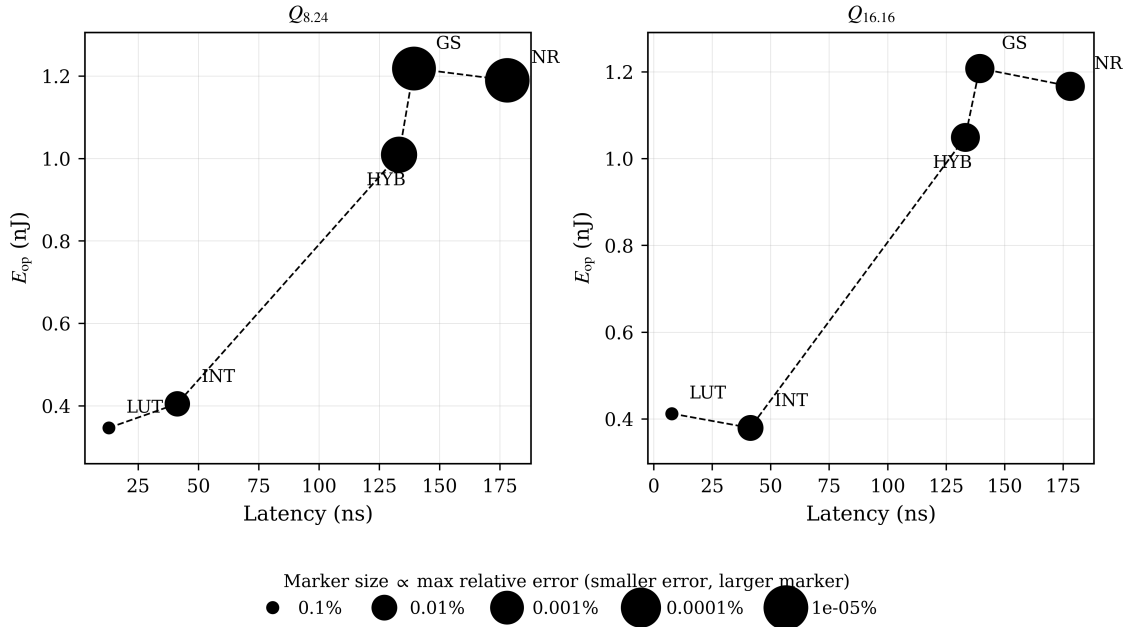


Fig. 26. Pareto trade-off in the latency-energy plane, by architecture, $Q_{8.24}$ (left) and $Q_{16.16}$ (right). Marker size is proportional to accuracy (max relative error) on a shared global scale across both panels; all five architectures are mutually non-dominated in both formats.

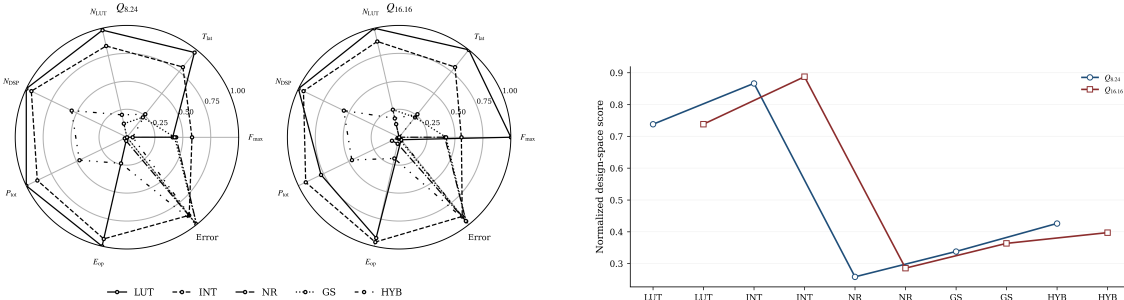


Fig. 27. Pareto score comparison for the evaluated architectures.

mean-reversion signal, where division by a continuously updated scale or standard-deviation term may be required; coordinate and vector normalization in graphics and radar/sonar signal chains, where a reciprocal can precede a subsequent multiplication; and fixed-point implementations of floating-point-adjacent arithmetic, where reciprocal computation forms part of mantissa normalization and division. In these settings, the appropriate architecture depends on the required combination of **numerical precision, latency, throughput, resource utilization, and energy**, with the evaluated designs occupying different regions of the corresponding trade-off space rather than a single architecture being universally optimal.

7.3 Limitations

Several limitations should be considered when interpreting and generalizing the results. First, all experiments use a single input domain, $x \in [0.5, 2.5]$. Although this range represents the normalized reciprocal domain considered in Section 2.A and captures the characteristic error concentration near the lower bound discussed in Section 5.3, the evaluation does not include alternative or non-uniform input distributions. Such distributions may alter the observed accuracy and performance trade-offs among the architectures.

Second, the evaluation is limited to a single FPGA family, Xilinx Artix-7. The relative costs of BRAM, DSP, LUT, and FF resources are device-dependent; consequently, the specific resource-utilization trade-offs reported in Table 3 may differ on other FPGA families or vendor platforms. The timing and power characteristics may likewise change with the underlying device architecture and implementation resources.

Third, the study does not include a direct comparison with vendor-supplied divider or reciprocal IP cores, such as Xilinx LogiCORE-class implementations, nor with CORDIC-based reciprocal/division computation [13, 14], which trades the multiplier-centric datapaths evaluated here for a shift-and-add rotation structure well suited to FPGA fabrics with limited DSP resources. Such comparisons would provide additional baselines for evaluating the custom architectures against off-the-shelf and multiplier-free solutions and are therefore left for future work.

Finally, the evaluated iterative architectures use fixed iteration configurations: **two Newton-Raphson iterations, two Goldschmidt iterations, and one Newton-Raphson refinement iteration following LUT interpolation in the hybrid architecture**. The seed-table sizes and interpolation resolutions are likewise fixed for the evaluated configurations. A broader exploration varying iteration count, seed-table size, and interpolation resolution could expose additional points in the accuracy, latency, resource, and energy trade-off space beyond the five architectures evaluated in this study.

8 Conclusion

This work presented a hardware-validated comparative evaluation of five fixed-point reciprocal approximation architectures—Direct LUT, LUT with Linear Interpolation, Newton-Raphson, Goldschmidt, and a hybrid LUT-Interpolation/Newton-Raphson architecture—implemented in Q8.24 and Q16.16 fixed-point formats on a Xilinx Artix-7 XC7A100T FPGA. All architectures were evaluated using a common input domain of $x \in [0.5, 2.5]$ and one million verification vectors, enabling accuracy, latency, resource utilization, and energy to be compared under a consistent experimental framework.

The results demonstrate substantial differences in the hardware and numerical characteristics of the evaluated architectures. The Direct LUT architecture provides the shortest pipelines, with latencies of 12.90 ns in Q8.24 and 7.74 ns in Q16.16, at the cost of the largest table-based memory requirement and the highest approximation error among the evaluated designs. LUT-based linear interpolation substantially reduces approximation error while retaining a relatively shallow pipeline, requiring 41.28 ns in both evaluated formats. The two iterative architectures provide substantially higher numerical accuracy but require deeper pipelined datapaths and significantly greater DSP and flip-flop utilization. In particular, the two-iteration Newton-Raphson architecture requires 69 pipeline cycles, whereas the two-iteration Goldschmidt architecture requires 54 cycles by exploiting parallelism between its intermediate numerator and denominator updates, reducing pipeline depth by approximately 22% relative to Newton-Raphson. The hybrid architecture combines a 9-bit interpolated seed with a single Newton-Raphson refinement iteration, providing an intermediate point in the accuracy, latency, DSP, and energy trade-off space.

The comparison also demonstrates that fixed-point format selection has a direct interaction with reciprocal architecture. Q8.24 generally preserves substantially higher numerical accuracy for the iterative and hybrid architectures because of its larger fractional field, whereas Q16.16 reduces fractional precision and consequently increases the observed approximation error for these architectures. At the same time, the change in fractional precision does not produce a uniform hardware advantage across all architectures, illustrating that arithmetic precision, memory organization, pipeline structure, and FPGA resource mapping must be considered together when selecting a fixed-point reciprocal implementation.

Across the evaluated designs, the maximum relative error spans more than three orders of magnitude, while latency spans more than an order of magnitude. Energy per operation varies by approximately 3.5 $\times$, from 0.346 nJ/op to 1.219 nJ/op. The three-objective Pareto analysis further shows that all five architectures remain mutually non-dominated within each fixed-point format: each occupies a distinct point in the accuracy-latency-energy design space rather than being uniformly superseded by another evaluated architecture. This result reinforces that reciprocal architecture selection is inherently application-dependent, with the appropriate implementation determined by the required balance among numerical precision, pipeline latency, throughput, energy, and FPGA resource availability.

Overall, the study provides a common hardware evaluation framework for understanding these trade-offs across lookup-based, interpolated, iterative, and hybrid reciprocal architectures. Rather than identifying a universally optimal implementation, the results establish quantitative design points that can be used to select or further optimize a reciprocal architecture according to the constraints of a target FPGA application. Future work can extend this design space to additional FPGA families, alternative input distributions, vendor-supplied reciprocal/division IP, and systematic variation of seed-table resolution and iteration count to investigate configurations beyond the five evaluated architectures.

9 Consolidated Results Table

Table 8 consolidates the key metrics from Tables 3-6 into a single reference table, for readers who want the complete set of headline figures for all ten architecture/format combinations without cross-referencing multiple tables.

Table 8. Consolidated Summary of All Reported Metrics, All Architectures and Formats

Architecture	Format	LUT%	DSP	Latency (ns)	Max Rel. Err (%)	Total P (W)	Energy/op (nJ)	EDP (nJ·ns)
Direct LUT	Q8.24	0.03%	0	12.90	0.097596	0.136	0.346	4.46
Direct LUT	Q16.16	0.003%	0	7.74	0.094787	0.214	0.412	3.19
LUT + Interp.	Q8.24	0.25%	1	41.28	0.010396	0.174	0.405	16.72
LUT + Interp.	Q16.16	0.19%	1	41.28	0.009156	0.161	0.379	15.66
Newton-Raphson	Q8.24	1.52%	20	178.02	0.000015	0.475	1.190	211.80
Newton-Raphson	Q16.16	1.34%	20	178.02	0.003815	0.458	1.167	207.68
Goldschmidt	Q8.24	1.33%	20	139.32	0.000020	0.483	1.219	169.79
Goldschmidt	Q16.16	1.14%	20	139.32	0.003814	0.477	1.208	168.24
Hybrid	Q8.24	1.21%	9	133.20	0.000409	0.319	1.009	134.44
Hybrid	Q16.16	1.26%	9	133.20	0.004118	0.320	1.049	139.73

Ethics and Privacy Statement

This work is a comparative hardware-architecture study of fixed-point arithmetic circuits on FPGAs and does not involve human subjects, personal data, or user-facing applications. The reciprocal approximation architectures evaluated

here are general-purpose arithmetic building blocks with no direct dual-use, surveillance, or safety-critical application inherent to the study itself; any downstream use, including in signal-processing or embedded systems, is subject to the standard engineering and safety review appropriate to that application domain. The authors are not aware of any societal risks specific to the publication of these results.

References

- [1] S. F. Oberman and M. J. Flynn, "Division Algorithms and Implementations," *IEEE Trans. Comput.*, vol. 46, no. 8, pp. 833-854, Aug. 1997, doi: 10.1109/12.609274.
- [2] D. Chen, B. Zhou, Z. Guo, and P. Nilsson, "Design and Implementation of Reciprocal Unit," *Proc. IEEE Midwest Symp. Circuits Syst. (MWSCAS)*, pp. 1318-1321, 2005, doi: 10.1109/MWSCAS.2005.1594352.
- [3] U. Kucukbakak and A. Akkas, "Design and Implementation of Reciprocal Unit Using Table Look-Up and Newton-Raphson Iteration," *Proc. Euromicro Symp. Digital System Design (DSD)*, pp. 249-253, 2004, doi: 10.1109/DSD.2004.1333284.
- [4] A. Rodríguez-García, L. Pizano-Escalante, R. Parra-Michel, O. Longoria-Gandara, and J. Cortez, "Fast Fixed-Point Divider Based on Newton-Raphson Method and Piecewise Polynomial Approximation," *Proc. Int. Conf. Reconfigurable Computing and FPGAs (ReConFig)*, 2013, doi: 10.1109/ReConFig.2013.6732291.
- [5] E. Libessart, M. Arzel, C. Lahuec, and F. P. Andriulli, "A Scaling-Less Newton-Raphson Pipelined Implementation for a Fixed-Point Reciprocal Operator," *IEEE Signal Process. Lett.*, vol. 24, no. 6, pp. 789-793, 2017, doi: 10.1109/LSP.2017.2694225.
- [6] D. Kumar, P. Saha, and A. Dandapat, "Hardware Implementation of Methodologies of Fixed Point Division Algorithms," *Int. J. Smart Sens. Intell. Syst.*, vol. 10, no. 3, 2017, doi: 10.21307/ijssis-2017-227.
- [7] M. F. Kasim, T. Adiono, M. Fahrza, and M. F. Zakiy, "FPGA Implementation of Fixed-Point Divider Using Pre-Computed Values," *Procedia Technology*, vol. 11, pp. 206-211, 2013, doi: 10.1016/j.protec.2013.12.182.
- [8] G. Paim, P. Marques, E. A. C. da Costa, S. J. M. de Almeida, and S. Bampi, "Improved Goldschmidt Algorithm for Fast and Energy-Efficient Fixed-Point Divider," *Proc. 24th IEEE Int. Conf. Electronics, Circuits and Systems (ICECS)*, pp. 482-485, 2017, doi: 10.1109/ICECS.2017.8292070.
- [9] G. Gutiérrez-Ramos, R. Parra-Michel, E. Romero-Aguirre, A. Rodríguez-García, and R. Jaramillo-Ramírez, "A Fast Fixed-Point Implementation for Division, Reciprocal, Square Root and Reciprocal Square Root Based on Newton-Raphson Method," *Electronics*, vol. 15, no. 13, Art. no. 2899, 2026, doi: 10.3390/electronics15132899.
- [10] R. E. Goldschmidt, "Applications of Division by Convergence," M.S. thesis, Dept. Electrical Engineering, Massachusetts Institute of Technology, Cambridge, MA, USA, 1964.
- [11] I. Kong and E. E. Swartzlander, Jr., "A Goldschmidt Division Method With Faster Than Quadratic Convergence," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 19, no. 4, pp. 696-700, Apr. 2011, doi: 10.1109/TVLSI.2009.2036926.
- [12] D. Das Sarma and D. W. Matula, "Faithful Bipartite ROM Reciprocal Tables," in *Proc. 12th IEEE Symp. Computer Arithmetic (ARITH)*, 1995, pp. 17-28, doi: 10.1109/ARITH.1995.465381.
- [13] J. E. Volder, "The CORDIC Trigonometric Computing Technique," *IRE Trans. Electron. Comput.*, vol. EC-8, no. 3, pp. 330-334, Sep. 1959, doi: 10.1109/TEC.1959.5222693.
- [14] R. Andraka, "A Survey of CORDIC Algorithms for FPGA Based Computers," in *Proc. 1998 ACM/SIGDA 6th Int. Symp. Field Programmable Gate Arrays (FPGA)*, 1998, pp. 191-200, doi: 10.1145/275107.275139.
- [15] M. D. Ercegovac and T. Lang, *Digital Arithmetic*. San Francisco, CA, USA: Morgan Kaufmann, 2004.
- [16] P. Soderquist and M. Leiser, "Area and Performance Tradeoffs in Floating-Point Divide and Square-Root Implementations," *ACM Comput. Surv.*, vol. 28, no. 3, pp. 518-564, Sep. 1996, doi: 10.1145/243439.243481.
- [17] M. D. Ercegovac and T. Lang, *Division and Square Root: Digit-Recurrence Algorithms and Implementations*. Boston, MA, USA: Kluwer Academic, 1994.
- [18] B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*. New York, NY, USA: Oxford Univ. Press, 2000.
- [19] J.-M. Muller, *Elementary Functions: Algorithms and Implementation*. Boston, MA, USA: Birkhäuser, 1997.
- [20] S. F. Anderson, J. G. Earle, R. E. Goldschmidt, and D. M. Powers, "The IBM System/360 Model 91: Floating-Point Execution Unit," *IBM J. Res. Develop.*, vol. 11, no. 1, pp. 34-53, Jan. 1967, doi: 10.1147/rd.111.0034.
- [21] M. J. Flynn, "On Division by Functional Iteration," *IEEE Trans. Comput.*, vol. C-19, no. 8, pp. 702-706, Aug. 1970, doi: 10.1109/T-C.1970.223026.
- [22] M. Ito, N. Takagi, and S. Yajima, "Efficient Initial Approximation for Multiplicative Division and Square Root by a Multiplication With Operand Modification," *IEEE Trans. Comput.*, vol. 46, no. 4, pp. 495-498, Apr. 1997, doi: 10.1109/12.588066.
- [23] M. J. Schulte and J. E. Stine, "Approximating Elementary Functions with Symmetric Bipartite Tables," *IEEE Trans. Comput.*, vol. 48, no. 8, pp. 842-847, Aug. 1999, doi: 10.1109/12.795125.
- [24] F. de Dinechin and A. Tisserand, "Multipartite Table Methods," *IEEE Trans. Comput.*, vol. 54, no. 3, pp. 319-330, Mar. 2005, doi: 10.1109/TC.2005.54.
- [25] M. D. Ercegovac, T. Lang, J.-M. Muller, and A. Tisserand, "Reciprocation, Square Root, Inverse Square Root, and Some Elementary Functions Using Small Multipliers," *IEEE Trans. Comput.*, vol. 49, no. 7, pp. 628-637, Jul. 2000, doi: 10.1109/12.863040.
- [26] J.-L. Beuchat and A. Tisserand, "Small Multiplier-Based Multiplication and Division Operators for Virtex-II Devices," in *Proc. 12th Int. Conf. Field-Programmable Logic and Applications (FPL)*, 2002, pp. 513-522, doi: 10.1007/3-540-46117-5_53.

- [27] J.-A. Piñeiro and J. D. Bruguera, "High-Speed Double-Precision Computation of Reciprocal, Division, Square Root, and Inverse Square Root," *IEEE Trans. Comput.*, vol. 51, no. 12, pp. 1377-1388, Dec. 2002, doi: 10.1109/TC.2002.1146704.
- [28] J.-A. Piñeiro, S. F. Oberman, J.-M. Muller, and J. D. Bruguera, "High-Speed Function Approximation Using a Minimax Quadratic Interpolator," *IEEE Trans. Comput.*, vol. 54, no. 3, pp. 304-318, Mar. 2005, doi: 10.1109/TC.2005.54.
- [29] M. D. Ercegovic, L. Imbert, D. W. Matula, J.-M. Muller, and G. Wei, "Improving Goldschmidt Division, Square Root, and Square Root Reciprocal," *IEEE Trans. Comput.*, vol. 49, no. 7, pp. 759-763, Jul. 2000, doi: 10.1109/12.863046.
- [30] W. L. Gallagher and E. E. Swartzlander, Jr., "Fault-Tolerant Newton-Raphson and Goldschmidt Dividers Using Time Shared TMR," *IEEE Trans. Comput.*, vol. 49, no. 6, pp. 588-595, Jun. 2000, doi: 10.1109/12.859542.
- [31] A. Nannarelli and T. Lang, "Power-Delay Tradeoffs for Radix-4 and Radix-8 Dividers," in *Proc. Int. Symp. Low Power Electronics and Design (ISLPED)*, 1998, pp. 109-111, doi: 10.1145/280756.280873.
- [32] X. Wang and B. Nelson, "Tradeoffs of Designing Floating-Point Division and Square Root on Virtex FPGAs," in *Proc. 11th Annu. IEEE Symp. Field-Programmable Custom Computing Machines (FCCM)*, Apr. 2003, pp. 195-203, doi: 10.1109/FPGA.2003.1227255.
- [33] N. Sorokin, "Implementation of High-Speed Fixed-Point Dividers on FPGA," *J. Comput. Sci. Technol.*, vol. 6, no. 1, pp. 8-11, Apr. 2006.
- [34] D. M. Muñoz, D. F. Sanchez, C. H. Llanos, and M. Ayala-Rincón, "Tradeoff of FPGA Design of a Floating-Point Library for Arithmetic Operators," *J. Integr. Circuits Syst.*, vol. 5, no. 1, pp. 42-52, 2010.
- [35] M. Lunglmayr, "Efficient Non-sequential Division for FPGAs," arXiv:2105.05747, 2021.
- [36] Xilinx, Inc., *7 Series FPGAs DSP48E1 Slice User Guide*, UG479 (v1.10), Mar. 2018.
- [37] N. Weaver, Y. Markovskiy, Y. Patel, and J. Wawrzyniek, "Post-Placement C-slow Retiming for the Xilinx Virtex FPGA," in *Proc. 11th ACM Int. Symp. Field-Programmable Gate Arrays (FPGA)*, Monterey, CA, USA, Feb. 2003, pp. 185-194, doi: 10.1145/611817.611844.
- [38] C. E. Leiserson and J. B. Saxe, "Retiming Synchronous Circuitry," *Algorithmica*, vol. 6, no. 1, pp. 5-35, 1991, doi: 10.1007/BF01759032.

Received XX October 2026; revised XX October 2026; accepted XX January 2027